In [1]:
```
import sys
!{sys.executable} -m pip install seaborn
```

```
Requirement already satisfied: seaborn in ./anaconda3/envs/mynev/lib/python
3.9/site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in ./anaconda3/envs/myne
v/lib/python3.9/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in ./anaconda3/envs/mynev/lib/pyt
hon3.9/site-packages (from seaborn) (2.2.1)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in ./anaconda3/envs/m
ynev/lib/python3.9/site-packages (from seaborn) (3.8.3)
Requirement already satisfied: contourpy>=1.0.1 in ./anaconda3/envs/mynev/li
b/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)
Requirement already satisfied: cycler>=0.10 in ./anaconda3/envs/mynev/lib/py
thon3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./anaconda3/envs/mynev/l
ib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.50.0)
Requirement already satisfied: kiwisolver>=1.3.1 in ./anaconda3/envs/mynev/l
ib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.5)
Requirement already satisfied: packaging>=20.0 in ./anaconda3/envs/mynev/li
b/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (23.1)
Requirement already satisfied: pillow>=8 in ./anaconda3/envs/mynev/lib/pytho
n3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in ./anaconda3/envs/mynev/li
b/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in ./anaconda3/envs/myne
v/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.8.
2)
Requirement already satisfied: importlib-resources>=3.2.0 in ./anaconda3/env
s/mynev/lib/python3.9/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(6.3.2)
Requirement already satisfied: pytz>=2020.1 in ./anaconda3/envs/mynev/lib/py
thon3.9/site-packages (from pandas>=1.2->seaborn) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.7 in ./anaconda3/envs/mynev/lib/
python3.9/site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: zipp>=3.1.0 in ./anaconda3/envs/mynev/lib/pyt
hon3.9/site-packages (from importlib-resources>=3.2.0->matplotlib!=3.6.1,>=
3.4->seaborn) (3.17.0)
Requirement already satisfied: six>=1.5 in ./anaconda3/envs/mynev/lib/python
3.9/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seabo
rn) (1.16.0)
```

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy.optimize import curve_fit
```

In [3]:
```
import mysql.connector as sql

conn=sql.connect(
    host='localhost',
    user='root',
    password='aa60726LX1',
```

```
    database='zybook',
    use_pure= True)

conn
```

Out[3]: <mysql.connector.connection.MySQLConnection at 0x13bb88880>

In [4]:
```
cursor=conn.cursor(buffered=True)
```

In [5]:
```
sql_query = "SELECT * FROM player"

# Fetch 'player' table into a DataFrame
player_data = pd.read_sql(sql_query, conn)
```

```
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/760777404.p
y:4: UserWarning: pandas only supports SQLAlchemy connectable (engine/connec
tion) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 obje
cts are not tested. Please consider using SQLAlchemy.
  player_data = pd.read_sql(sql_query, conn)
```

In [6]:
```
cursor.execute("show tables")
for x in cursor:
    print(x)
```

```
('appearances',)
('city',)
('country_language',)
('fielding',)
('fielding_outfield',)
('fielding_postseason',)
('flight_data',)
('house_data_clf',)
('house_data_reg',)
('Movie',)
('player',)
('salary',)
```

In [7]:
```
player_data
```

Out[7]:

	player_id	birth_year	name_first	name_last	name_given	weight	height	ba
0	0	1981			David	0	0	
1	aardsda01	1981	David	Aardsma	David Allan	220	75	
2	aaronha01	1934	Hank	Aaron	Henry Louis	180	72	
3	aaronto01	1939	Tommie	Aaron	Tommie Lee	190	75	
4	aasedo01	1954	Don	Aase	Donald William	190	75	
...	...	...	...	...	...	...	...	
17721	zupofr01	1939	Frank	Zupo	Frank Joseph	182	71	
17722	zuvelpa01	1958	Paul	Zuvella	Paul	173	72	
17723	zuverge01	1924	George	Zuverink	George	195	76	
17724	zwilldu01	1888	Dutch	Zwilling	Edward Harrison	160	66	
17725	zychto01	1990	Tony	Zych	Anthony Aaron	190	75	

17726 rows × 11 columns

In [8]:
```
sql_query = "SELECT * FROM fielding"

# Fetch 'player' table into a DataFrame
fielding_data = pd.read_sql(sql_query, conn)
```

```
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2328175338.py:4: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  fielding_data = pd.read_sql(sql_query, conn)
```

In [9]:
```
fielding_data
```

Out[9]:

	player_id	year	stint	team_id	league_id	pos	g	gs	inn_outs	putouts
0	aardsda01	2004	1	SFN	NL	P	11		33.0	0
1	aardsda01	2006	1	CHN	NL	P	45		159.0	1
2	aardsda01	2007	1	CHA	AL	P	25		96.0	2
3	aardsda01	2008	1	BOS	AL	P	47		147.0	3
4	aardsda01	2009	1	SEA	AL	P	73		213.0	2
...	...	...	...	...	...	...	...	...	...	...
38736	zuvelpa01	1987	1	NYA	AL	2B	7	6.0	168.0	17
38737	zuvelpa01	1988	1	CLE	AL	SS	49	44.0	1139.0	77
38738	zuvelpa01	1989	1	CLE	AL	3B	5	5.0	131.0	4
38739	zuvelpa01	1991	1	KCA	AL	3B	2	0.0	6.0	0
38740	zychto01	2015	1	SEA	AL	P	13			0

38741 rows × 18 columns

In [10]:
```
player_data
```

Out[10]:

	player_id	birth_year	name_first	name_last	name_given	weight	height	ba
0	0	1981			David	0	0	
1	aardsda01	1981	David	Aardsma	David Allan	220	75	
2	aaronha01	1934	Hank	Aaron	Henry Louis	180	72	
3	aaronto01	1939	Tommie	Aaron	Tommie Lee	190	75	
4	aasedo01	1954	Don	Aase	Donald William	190	75	
...	...	...	...	...	...	...	...	
17721	zupofr01	1939	Frank	Zupo	Frank Joseph	182	71	
17722	zuvelpa01	1958	Paul	Zuvella	Paul	173	72	
17723	zuverge01	1924	George	Zuverink	George	195	76	
17724	zwilldu01	1888	Dutch	Zwilling	Edward Harrison	160	66	
17725	zychto01	1990	Tony	Zych	Anthony Aaron	190	75	

17726 rows × 11 columns

In [11]:
```
player_data.columns
```

Out[11]:
```
Index(['player_id', 'birth_year', 'name_first', 'name_last', 'name_given',
       'weight', 'height', 'bats', 'throws', 'retro_id', 'bbref_id'],
      dtype='object')
```

In [12]:
```
#get rid of unnecessary columns
player_data.drop(columns=['name_first', 'name_last'], inplace=True)
player_data
```

Out[12]:

	player_id	birth_year	name_given	weight	height	bats	throws	retro_id	
0	0	1981	David	0	0	2	7	2004-04-06	2
1	aardsda01	1981	David Allan	220	75	R	R	aardd001	aa
2	aaronha01	1934	Henry Louis	180	72	R	R	aaroh101	aa
3	aaronto01	1939	Tommie Lee	190	75	R	R	aarot101	a
4	aasedo01	1954	Donald William	190	75	R	R	aased001	a
...	...	...	...	...	...	...	...	...	
17721	zupofr01	1939	Frank Joseph	182	71	L	R	zupof101	
17722	zuvelpa01	1958	Paul	173	72	R	R	zuvep001	z
17723	zuverge01	1924	George	195	76	R	R	zuveg101	zu
17724	zwilldu01	1888	Edward Harrison	160	66	L	L	zwild101	z
17725	zychto01	1990	Anthony Aaron	190	75	R	R	zycht001	z

17726 rows × 9 columns

In [13]:
```
#join data to get player info (ages)on the table
joined_data = pd.merge(fielding_data, player_data, on='player_id')


# Print first few rows of joined_data
print("Joined Data:")
print(joined_data.head())
joined_data_copy = joined_data.copy()
```

```
Joined Data:
   player_id  year  stint team_id league_id pos   g gs inn_outs  putouts
...  \
0  aardsda01  2004      1     SFN        NL   P  11        33.0        0
...
1  aardsda01  2006      1     CHN        NL   P  45       159.0        1
...
2  aardsda01  2007      1     CHA        AL   P  25        96.0        2
...
3  aardsda01  2008      1     BOS        AL   P  47       147.0        3
...
4  aardsda01  2009      1     SEA        AL   P  73       213.0        2
...

   caught_stealing  zone_rating  birth_year   name_given weight height bats
\
0                                       1981  David Allan    220     75    R
1                                       1981  David Allan    220     75    R
2                                       1981  David Allan    220     75    R
3                                       1981  David Allan    220     75    R
4                                       1981  David Allan    220     75    R

  throws  retro_id   bbref_id
0      R  aardd001  aardsda01
1      R  aardd001  aardsda01
2      R  aardd001  aardsda01
3      R  aardd001  aardsda01
4      R  aardd001  aardsda01

[5 rows x 26 columns]
```

In [14]:
```
joined_data.columns
```

Out[14]:
```
Index(['player_id', 'year', 'stint', 'team_id', 'league_id', 'pos', 'g', 'g
s',
       'inn_outs', 'putouts', 'assists', 'errors', 'double_plays',
       'passed_balls', 'wild_pitches', 'opponent_stolen_bases',
       'caught_stealing', 'zone_rating', 'birth_year', 'name_given', 'weigh
t',
       'height', 'bats', 'throws', 'retro_id', 'bbref_id'],
      dtype='object')
```

In [15]:
```
#remove unnecessary player data from table
columns_to_drop = ['weight', 'height', 'bats', 'throws', 'retro_id', 'bbref_
joined_data.drop(columns=columns_to_drop, inplace=True)
```

In [16]:
```
joined_data
```

Out[16]:

	player_id	year	stint	team_id	league_id	pos	g	gs	inn_outs	putouts
0	aardsda01	2004	1	SFN	NL	P	11		33.0	0
1	aardsda01	2006	1	CHN	NL	P	45		159.0	1
2	aardsda01	2007	1	CHA	AL	P	25		96.0	2
3	aardsda01	2008	1	BOS	AL	P	47		147.0	3
4	aardsda01	2009	1	SEA	AL	P	73		213.0	2
...	...	...	...	...	...	...	...	...	...	...
38736	zuvelpa01	1987	1	NYA	AL	2B	7	6.0	168.0	17
38737	zuvelpa01	1988	1	CLE	AL	SS	49	44.0	1139.0	77
38738	zuvelpa01	1989	1	CLE	AL	3B	5	5.0	131.0	4
38739	zuvelpa01	1991	1	KCA	AL	3B	2	0.0	6.0	0
38740	zychto01	2015	1	SEA	AL	P	13			0

38741 rows × 20 columns

In [17]:

```
#assign player ages to the stats from that season
joined_data['age'] = joined_data['year'] - joined_data['birth_year']

# Print first few rows of joined_data
print("Joined Data with 'age' column:")
print(joined_data.head())
```

```
Joined Data with 'age' column:
   player_id  year  stint team_id league_id pos   g gs inn_outs  putouts
...  \
0  aardsda01  2004      1     SFN        NL   P  11        33.0        0
...
1  aardsda01  2006      1     CHN        NL   P  45       159.0        1
...
2  aardsda01  2007      1     CHA        AL   P  25        96.0        2
...
3  aardsda01  2008      1     BOS        AL   P  47       147.0        3
...
4  aardsda01  2009      1     SEA        AL   P  73       213.0        2
...

   errors  double_plays  passed_balls wild_pitches opponent_stolen_bases  \
0       0             0
1       0             1
2       1             0
3       0             0
4       0             1

  caught_stealing zone_rating birth_year   name_given age
0                                   1981  David Allan  23
1                                   1981  David Allan  25
2                                   1981  David Allan  26
3                                   1981  David Allan  27
4                                   1981  David Allan  28

[5 rows x 21 columns]
```

In [18]:
```
#remove rows with data older than 1985
joined_data = joined_data[joined_data['year'] >= 1985]
```

In [19]:
```
joined_data
```

Out[19]:

	player_id	year	stint	team_id	league_id	pos	g	gs	inn_outs	putouts
0	aardsda01	2004	1	SFN	NL	P	11		33.0	0
1	aardsda01	2006	1	CHN	NL	P	45		159.0	1
2	aardsda01	2007	1	CHA	AL	P	25		96.0	2
3	aardsda01	2008	1	BOS	AL	P	47		147.0	3
4	aardsda01	2009	1	SEA	AL	P	73		213.0	2
...	...	...	...	...	...	...	...	...	...	...
38736	zuvelpa01	1987	1	NYA	AL	2B	7	6.0	168.0	17
38737	zuvelpa01	1988	1	CLE	AL	SS	49	44.0	1139.0	77
38738	zuvelpa01	1989	1	CLE	AL	3B	5	5.0	131.0	4
38739	zuvelpa01	1991	1	KCA	AL	3B	2	0.0	6.0	0
38740	zychto01	2015	1	SEA	AL	P	13			0

38741 rows × 21 columns

In [20]:
```
joined_data.columns
```

Out[20]:
```
Index(['player_id', 'year', 'stint', 'team_id', 'league_id', 'pos', 'g', 'g
s',
       'inn_outs', 'putouts', 'assists', 'errors', 'double_plays',
       'passed_balls', 'wild_pitches', 'opponent_stolen_bases',
       'caught_stealing', 'zone_rating', 'birth_year', 'name_given', 'ag
e'],
      dtype='object')
```

In [21]:
```
modern_joined_data=joined_data.copy()
```

In [22]:
```
#clean data and remove unnecessary columns
modern_joined_data.drop(columns=['stint', 'team_id', 'league_id', 'player_ic
```

In [23]:
```
modern_joined_data
```

Out[23]:

	pos	g	gs	inn_outs	putouts	assists	errors	double_plays	passed_balls	...
0	P	11		33.0	0	0	0	0		
1	P	45		159.0	1	5	0	1		
2	P	25		96.0	2	4	1	0		
3	P	47		147.0	3	6	0	0		
4	P	73		213.0	2	5	0	1		
...	...	...	...	...	...	...	...	...	...	...
38736	2B	7	6.0	168.0	17	18	0	5		
38737	SS	49	44.0	1139.0	77	112	8	21		
38738	3B	5	5.0	131.0	4	8	1	0		
38739	3B	2	0.0	6.0	0	0	0	0		
38740	P	13			0	3	0	0		

38741 rows × 15 columns

In [24]:
```
position_order = ['C', 'P', '1B', '2B', '3B', 'SS', 'LF', 'CF', 'RF','DH','C

# Sort the DataFrame based on the 'pos' column using the specified order
fielding_data_sorted = modern_joined_data.sort_values(by='pos', key=lambda x
print("Sorted Fielding Data:")
print(fielding_data_sorted.head())
```

```
Sorted Fielding Data:
      pos   g gs inn_outs  putouts  assists  errors  double_plays  \
19370   C   9                 55        3       1             1
20810   C  64      1290.0      267       32       4             3
20811   C   2         6.0        2        0       0             0
20815   C   3        21.0        4        0       1             0
20816   C   1         3.0        0        0       0             0

      passed_balls wild_pitches opponent_stolen_bases caught_stealing  \
19370          1.0                                3.0             2.0
20810          2.0                               36.0            18.0
20811          0.0                                0.0             0.0
20815          0.0                                1.0             0.0
20816          0.0                                0.0             0.0

      zone_rating     name_given  age
19370                 Ryan Cole   28
20810             Fernando Jose   24
20811             Fernando Jose   25
20815               James Lewis   23
20816               James Lewis   24
```

In [25]:
```
#seperate catchers from the rest of the data
# Filter out instances with position 'C'
```

```
c_data = fielding_data_sorted[fielding_data_sorted['pos'] == 'C']

# Remove 'C' instances from fielding_data_sorted
fielding_data_sorted = fielding_data_sorted[fielding_data_sorted['pos'] != '

# Print first few rows of c_data
print("C Data:")
print(c_data.head())

# Print first few rows of fielding_data_sorted after removing 'C'
print("\nRemaining Fielding Data:")
print(fielding_data_sorted.head())
```

```
C Data:
      pos   g gs inn_outs  putouts  assists  errors  double_plays  \
19370   C   9                  55        3       1             1
20810   C  64      1290.0      267       32       4             3
20811   C   2         6.0        2        0       0             0
20815   C   3        21.0        4        0       1             0
20816   C   1         3.0        0        0       0             0

      passed_balls wild_pitches opponent_stolen_bases caught_stealing  \
19370          1.0                                3.0             2.0
20810          2.0                               36.0            18.0
20811          0.0                                0.0             0.0
20815          0.0                                1.0             0.0
20816          0.0                                0.0             0.0

      zone_rating     name_given  age
19370                  Ryan Cole   28
20810              Fernando Jose   24
20811              Fernando Jose   25
20815                James Lewis   23
20816                James Lewis   24

Remaining Fielding Data:
      pos   g   gs inn_outs  putouts  assists  errors  double_plays  \
15798   P  65         189.0        6        5       0             2
15438   P  20          51.0        0        5       0             0
15799   P  61         171.0        3        4       0             0
23359   P  35  3.0    310.0        9       17       0             2
15437   P   5          54.0        2        7       0             1

      passed_balls wild_pitches opponent_stolen_bases caught_stealing  \
15798
15438
15799
23359
15437

      zone_rating      name_given  age
15798              Trevor William   39
15438                      Yoslan   33
15799              Trevor William   40
23359                Alan Bernard   26
15437                      Yoslan   27
```

In [26]:
```
#remove non-fielder playing stats
fielding_data_sorted=fielding_data_sorted.drop(columns=['passed_balls','wild
```

In [27]:
```
#add score column

fielding_data_sorted['score'] = 0

# Print first few rows of fielding_data_sorted with the new column
print("Fielding Data with 'score' column:")
print(fielding_data_sorted.head())
```

```
Fielding Data with 'score' column:
       pos   g   gs inn_outs  putouts  assists  errors  double_plays  \
15798   P  65          189.0        6        5       0             2
15438   P  20           51.0        0        5       0             0
15799   P  61          171.0        3        4       0             0
23359   P  35  3.0     310.0        9       17       0             2
15437   P   5           54.0        2        7       0             1

            name_given  age  score
15798  Trevor William   39      0
15438          Yoslan   33      0
15799  Trevor William   40      0
23359    Alan Bernard   26      0
15437          Yoslan   27      0
```

In [28]:
```
# Replace empty strings with NaN
fielding_data_sorted['inn_outs'].replace('', np.nan, inplace=True)

# Convert the 'inn_outs' column to float
fielding_data_sorted['inn_outs'] = fielding_data_sorted['inn_outs'].astype(f
```

```
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/1202444388.p
y:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  fielding_data_sorted['inn_outs'].replace('', np.nan, inplace=True)
```

In [29]:
```
# Calculate scores for position players
# Define a function for safe division with error handling
def safe_division(numerator, denominator):
    # Handle case when denominator is 0 or NaN
    return np.where((denominator == 0) | np.isnan(denominator), np.nan, nume

# Calculate the score based on the formula and handle division by zero
fielding_data_sorted['score'] = safe_division((fielding_data_sorted['putouts

# Print first few rows of fielding_data_sorted with the new 'score' column
```

```
print("Fielding Data with 'score' column calculated:")
print(fielding_data_sorted.head())
```

```
Fielding Data with 'score' column calculated:
      pos   g   gs  inn_outs  putouts  assists  errors  double_plays  \
15798   P  65          189.0        6        5       0             2
15438   P  20           51.0        0        5       0             0
15799   P  61          171.0        3        4       0             0
23359   P  35  3.0     310.0        9       17       0             2
15437   P   5           54.0        2        7       0             1

           name_given  age     score
15798  Trevor William   39  0.058201
15438          Yoslan   33  0.098039
15799  Trevor William   40  0.040936
23359    Alan Bernard   26  0.083871
15437          Yoslan   27  0.166667
```

In [30]:
```
c_data_dtypes = c_data.dtypes

print("Data types for each column of c_data:")
print(c_data_dtypes)
```

```
Data types for each column of c_data:
pos                     object
g                        int64
gs                      object
inn_outs                object
putouts                  int64
assists                  int64
errors                   int64
double_plays             int64
passed_balls            object
wild_pitches            object
opponent_stolen_bases   object
caught_stealing         object
zone_rating             object
name_given              object
age                      int64
dtype: object
```

In [31]:
```
c_data['opponent_stolen_bases'] = c_data['opponent_stolen_bases'].astype(flo

# Convert 'passed_balls' column to integer
c_data['passed_balls'] = c_data['passed_balls'].astype(float)

# Replace empty strings with NaN
c_data['inn_outs'].replace('', np.nan, inplace=True)

# Convert the 'inn_outs' column to float
c_data['inn_outs'] = c_data['inn_outs'].astype(float)
```

```
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/772531906.p
y:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  c_data['inn_outs'].replace('', np.nan, inplace=True)
```

In [32]:
```
import numpy as np

# Define a function for safe division with error handling
def safe_division(numerator, denominator):
    # Handle case when denominator is 0 or NaN
    return np.where((denominator == 0) | np.isnan(denominator), np.nan, nume

# Calculate the score based on the formula and handle division by zero
c_data['score'] = safe_division(c_data['assists'] + c_data['putouts'] - c_da

# Print first few rows of c_data with the new 'score' column
print("C Data with 'score' column calculated:")
print(c_data.head())
```

```
C Data with 'score' column calculated:
      pos   g gs  inn_outs  putouts  assists  errors  double_plays  \
19370   C   9          NaN       55        3       1             1
20810   C  64       1290.0      267       32       4             3
20811   C   2          6.0        2        0       0             0
20815   C   3         21.0        4        0       1             0
20816   C   1          3.0        0        0       0             0

       passed_balls wild_pitches  opponent_stolen_bases caught_stealing  \
19370           1.0                                3.0             2.0
20810           2.0                               36.0            18.0
20811           0.0                                0.0             0.0
20815           0.0                                1.0             0.0
20816           0.0                                0.0             0.0

      zone_rating     name_given  age     score
19370                  Ryan Cole   28       NaN
20810              Fernando Jose   24  0.202326
20811              Fernando Jose   25  0.333333
20815                James Lewis   23  0.142857
20816                James Lewis   24  0.000000
```

In [33]:
```
# Find maximum and minimum scores for c_data
max_score_c = c_data['score'].max()
min_score_c = c_data['score'].min()

# Find maximum and minimum scores for fielding_data_sorted
max_score_fielding = fielding_data_sorted['score'].max()
```

```
min_score_fielding = fielding_data_sorted['score'].min()

# Print the results
print("Maximum score for c_data:", max_score_c)
print("Minimum score for c_data:", min_score_c)

print("\nMaximum score for fielding_data_sorted:", max_score_fielding)
print("Minimum score for fielding_data_sorted:", min_score_fielding)
```

```
Maximum score for c_data: 1.0
Minimum score for c_data: -0.3333333333333333

Maximum score for fielding_data_sorted: 3.3333333333333335
Minimum score for fielding_data_sorted: -0.5
```

In [34]:
```
#ensuring no calculation errors on the extreme
max_score = fielding_data_sorted['score'].max()

# Filter the DataFrame to show rows with the maximum score
max_score_rows = fielding_data_sorted[fielding_data_sorted['score'] == max_s

# Print the rows with the maximum score
print("Rows with the maximum score from fielding_data_sorted:")
print(max_score_rows)
```

```
Rows with the maximum score from fielding_data_sorted:
      pos  g   gs  inn_outs  putouts  assists  errors  double_plays  \
27512  1B  1  0.0       3.0        9        1       0             1

          name_given  age     score
27512  Keith Anthony   30  3.333333
```

In [35]:
```
fielding_data_sorted.rename(columns={'pos': 'position'}, inplace=True)
```

In [36]:
```
# Separate DataFrames for each position
p_data = fielding_data_sorted[fielding_data_sorted['position'] == 'P']

b1_data = fielding_data_sorted[fielding_data_sorted['position'] == '1B']
b2_data = fielding_data_sorted[fielding_data_sorted['position'] == '2B']
b3_data = fielding_data_sorted[fielding_data_sorted['position'] == '3B']
ss_data = fielding_data_sorted[fielding_data_sorted['position'] == 'SS']

# Group LF, RF, CF into OF and remove DH
of_data = fielding_data_sorted[fielding_data_sorted['position'].isin(['LF',
of_data = of_data[of_data['position'] != 'DH']

# Print first few rows of each DataFrame
print("P Data:")
print(p_data.head())

print("\n1B Data:")
print(b1_data.head())

print("\n2B Data:")
print(b2_data.head())

print("\n3B Data:")
```

```
print(b3_data.head())

print("\nSS Data:")
print(ss_data.head())

print("\nOF Data:")
print(of_data.head())
```

```
P Data:
      position   g   gs  inn_outs  putouts  assists  errors  double_plays  \
15798        P  65          189.0        6        5       0             2
15438        P  20           51.0        0        5       0             0
15799        P  61          171.0        3        4       0             0
23359        P  35  3.0     310.0        9       17       0             2
15437        P   5           54.0        2        7       0             1

          name_given  age     score
15798  Trevor William   39  0.058201
15438          Yoslan   33  0.098039
15799  Trevor William   40  0.040936
23359    Alan Bernard   26  0.083871
15437          Yoslan   27  0.166667

1B Data:
      position    g   gs  inn_outs  putouts  assists  errors  double_plays
\
25635       1B    2  0.0       9.0        2        1       0             0
32850       1B   10          231.0       63        4       1             9
8120        1B    2           30.0        9        1       0             0
32841       1B  135         3045.0      939       79       7           104
25632       1B    2  0.0      12.0        6        0       0             1

            name_given  age     score
25635     Joseph Melton   33  0.333333
32850   Bradley Michael   32  0.285714
8120   Mark Christopher   24  0.333333
32841       Jack Thomas   34  0.332020
25632     Joseph Melton   30  0.500000

2B Data:
      position   g    gs  inn_outs  putouts  assists  errors  double_plays
\
175         2B   2             NaN        4        7       0             1
5498        2B  24  22.0     590.0       56       73       6            20
14860       2B   8           213.0       17       20       0             4
174         2B   2            30.0        2        6       0             2
29698       2B  68  64.0    1690.0      128      185       7            35

             name_given  age     score
175    Cristhian Pascual   24       NaN
5498              Ramon   26  0.208475
14860            Adeiny   23  0.173709
174    Cristhian Pascual   23  0.266667
29698        Leon Joseph   28  0.181065

3B Data:
      position    g     gs  inn_outs  putouts  assists  errors  double_plays
\
34658       3B  150  144.0    3835.0       86      262      17            24
16796       3B   16   13.0     367.0        6       38       2             4
34670       3B    1              0.0        0        0       0             0
3333        3B  123  120.0    3077.0       62      201       7            24
34657       3B  134  128.0    3434.0       75      214      16            22
```

```
         name_given  age     score
34658  James Howard   26  0.086310
16796   Charles Leo   24  0.114441
34670  James Howard   41       NaN
3333   Wade Anthony   38  0.083198
34657  James Howard   25  0.079499

SS Data:
      position    g gs  inn_outs  putouts  assists  errors  double_plays  \
20783       SS   32        729.0       39       51       7             8
19276       SS  135       3270.0      191      370      12            89
20778       SS  156       4017.0      310      424      24            94
19275       SS   44       1023.0       65      108       9            23
19274       SS  102       2535.0      153      249      11            43

        name_given  age     score
20783  Julio Cesar   34  0.113855
19276  Barry Louis   38  0.167890
20778  Julio Cesar   30  0.176749
19275  Barry Louis   37  0.160313
19274  Barry Louis   36  0.154241

OF Data:
      position    g     gs  inn_outs  putouts  assists  errors  double_plays
\
21539       LF  134  126.0    3314.0      196        3      10             0
995         LF    2    0.0       6.0        0        0       0             0
21538       LF  114  110.0    2767.0      192        6       3             1
21532       LF    7    1.0      63.0        6        0       0             0
21537       LF  110  107.0    2789.0      125        8       6             1

           name_given  age     score
21539      Albert Lee   32  0.057031
995    Antonio Rafael   34  0.000000
21538      Albert Lee   31  0.070473
21532      Albert Lee   25  0.095238
21537      Albert Lee   30  0.045536
```

In [37]:

```python
summary_stats = {}

# Compute summary statistics for each position
for position, df in {'C': c_data, 'P': p_data, '1B': b1_data, '2B': b2_data,
    summary_stats[position] = df['score'].describe()

# Create a DataFrame from the summary statistics
summary_df = pd.DataFrame(summary_stats)

# Transpose for better readability
summary_df = summary_df.T

# Display the summary statistics
print("Summary Statistics for Each Position:")
print(summary_df)
```

```
Summary Statistics for Each Position:
      count      mean       std       min       25%       50%       75%  \
C    2460.0  0.240032  0.070923 -0.333333  0.208969  0.237190  0.266667
P   18095.0  0.059498  0.042496 -0.500000  0.037612  0.056911  0.076923
1B   5032.0  0.342166  0.114090 -0.333333  0.315667  0.340692  0.371475
2B   3377.0  0.176780  0.070016 -0.333333  0.160000  0.179253  0.198693
3B   1306.0  0.087599  0.055265 -0.500000  0.077700  0.092016  0.103133
SS    980.0  0.160863  0.039290 -0.111111  0.150306  0.161926  0.173846
OF   5732.0  0.083827  0.041453 -0.333333  0.069037  0.082841  0.097102

         max
C   1.000000
P   1.000000
1B  3.333333
2B  1.000000
3B  0.555556
SS  0.666667
OF  1.000000
```

In [38]:

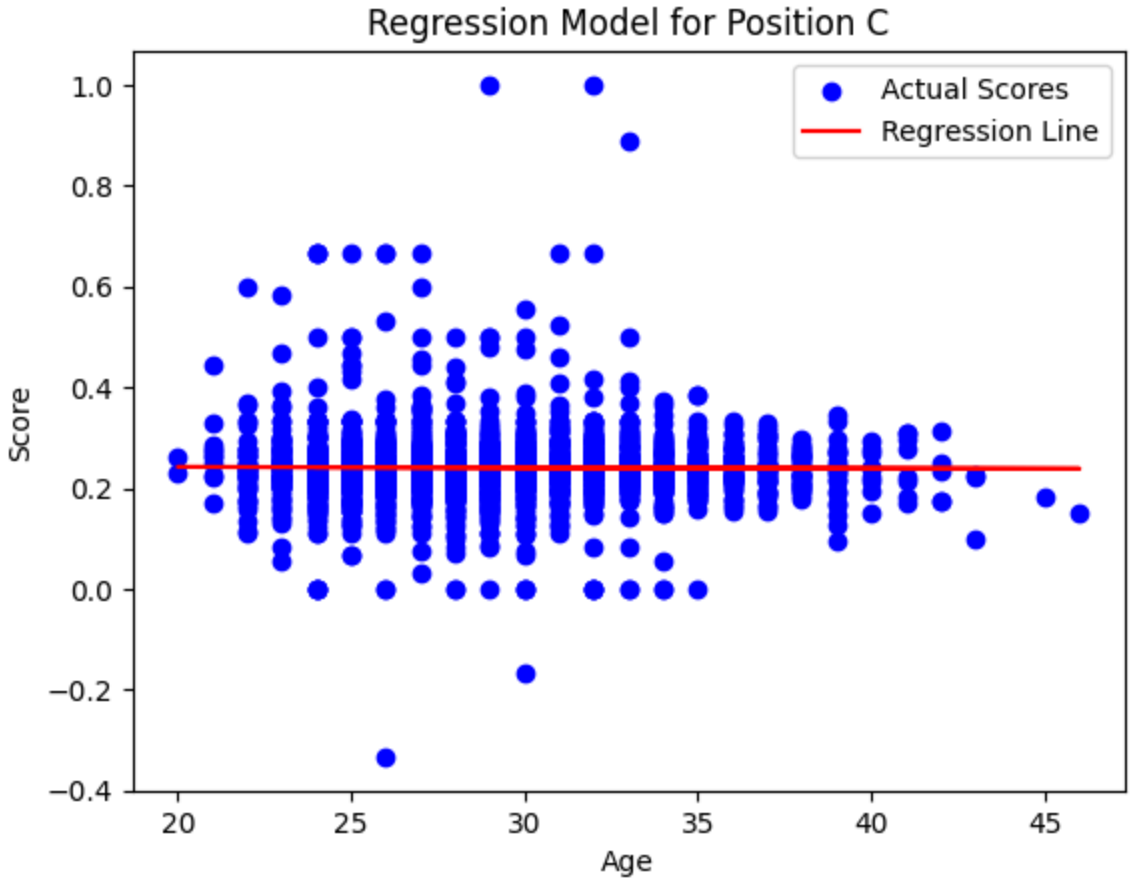
```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Define a function to create and evaluate a regression model for each posit
def create_and_evaluate_regression_model(data, position):
    # Make a copy of the data to avoid modifying the original DataFrame
    data_copy = data.copy()

    # Drop rows with missing values
    data_copy.dropna(subset=['score', 'age'], inplace=True)

    # Extract features (X) and target variable (y)
    X = data_copy[['age']]  # 'age' is the feature
    y = data_copy['score']  # 'score' is the target variable

    # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

    # Create a Linear Regression model
    model = LinearRegression()

    # Fit the model to the training data
    model.fit(X_train, y_train)

    # Evaluate the model on the testing data
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)

    # Print model coefficients and evaluation metric
    print(f"Regression coefficients for {position}:")
    print("Intercept:", model.intercept_)
    print("Coefficient:", model.coef_[0])
    print("Mean Squared Error:", mse)

    # Return the trained model
    return model
```

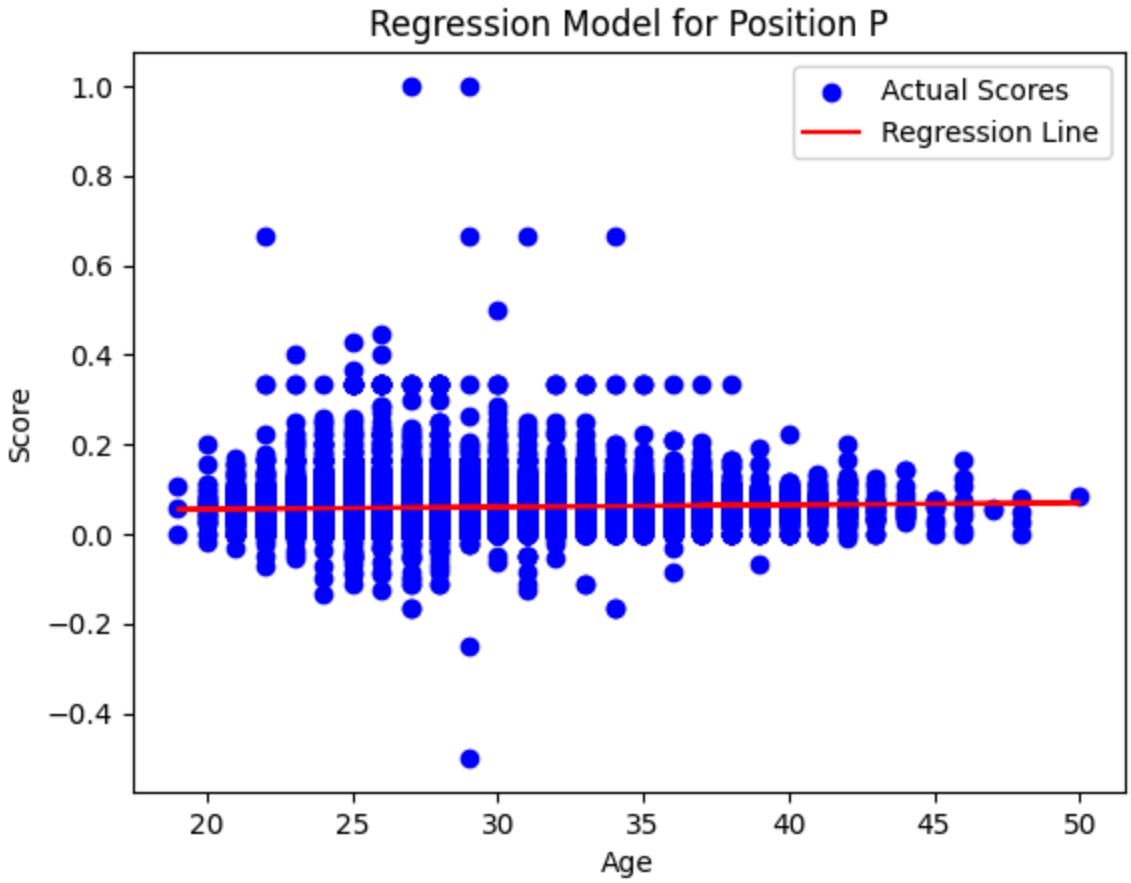
```
# Create and evaluate regression models for each position
positions = ['C', 'P', '1B', '2B', '3B', 'SS', 'OF']
models = {}

for position, data in zip(positions, [c_data, p_data, b1_data, b2_data, b3_d
    print(f"Position: {position}")
    models[position] = create_and_evaluate_regression_model(data, position)
    print("\n")
```

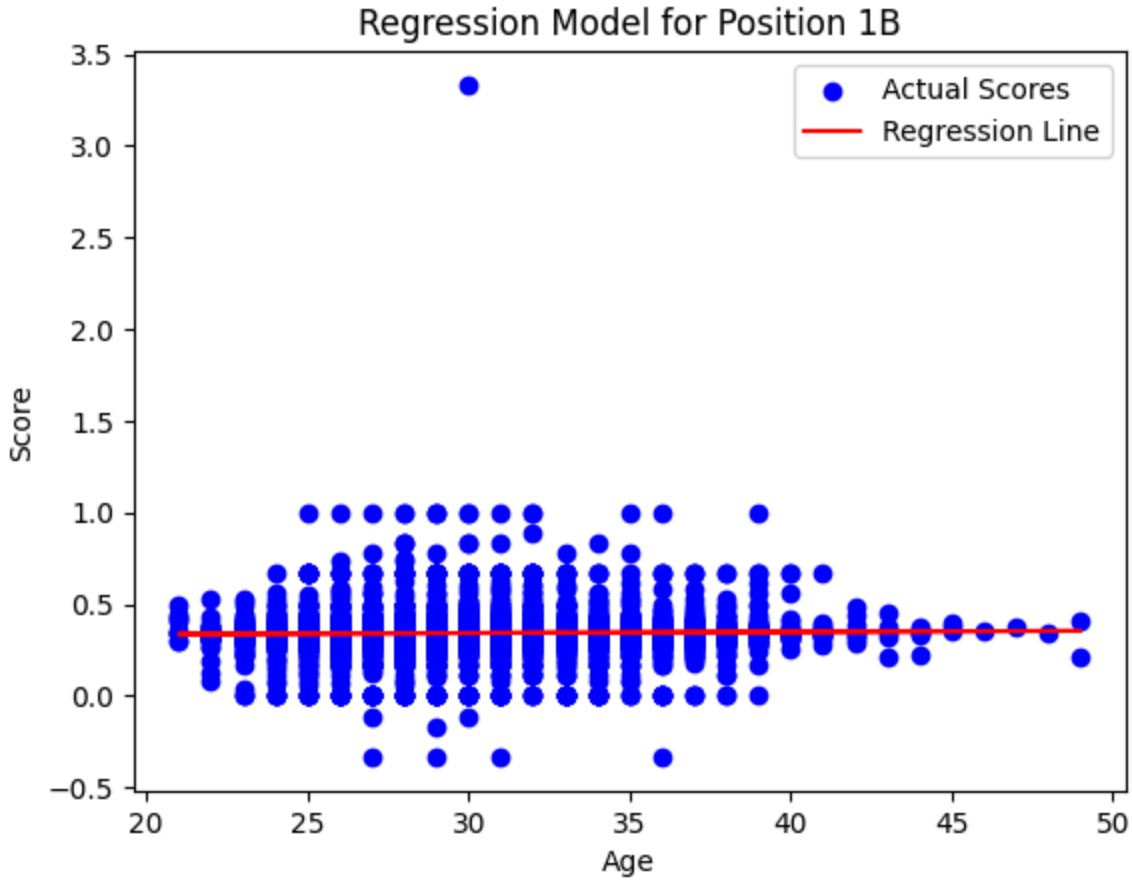
```
Position: C
Regression coefficients for C:
Intercept: 0.24466697484431654
Coefficient: -0.00013429095451952962
Mean Squared Error: 0.004515489990152277


Position: P
Regression coefficients for P:
Intercept: 0.04591098082215887
Coefficient: 0.0004703647519776519
Mean Squared Error: 0.0017200530047111002


Position: 1B
Regression coefficients for 1B:
Intercept: 0.32284113433594536
Coefficient: 0.0006424635120882252
Mean Squared Error: 0.012269141572367099


Position: 2B
Regression coefficients for 2B:
Intercept: 0.16391298464012868
Coefficient: 0.0004575888840130101
Mean Squared Error: 0.0034787890116638387


Position: 3B
Regression coefficients for 3B:
Intercept: 0.1091538367962494
Coefficient: -0.0007930734379335808
Mean Squared Error: 0.0033621675409385788


Position: SS
Regression coefficients for SS:
Intercept: 0.14686932662355862
Coefficient: 0.00048718293938758856
Mean Squared Error: 0.0009519096966920087


Position: OF
Regression coefficients for OF:
Intercept: 0.10273321551242943
Coefficient: -0.0006452441961604442
Mean Squared Error: 0.0014021912604429484
```

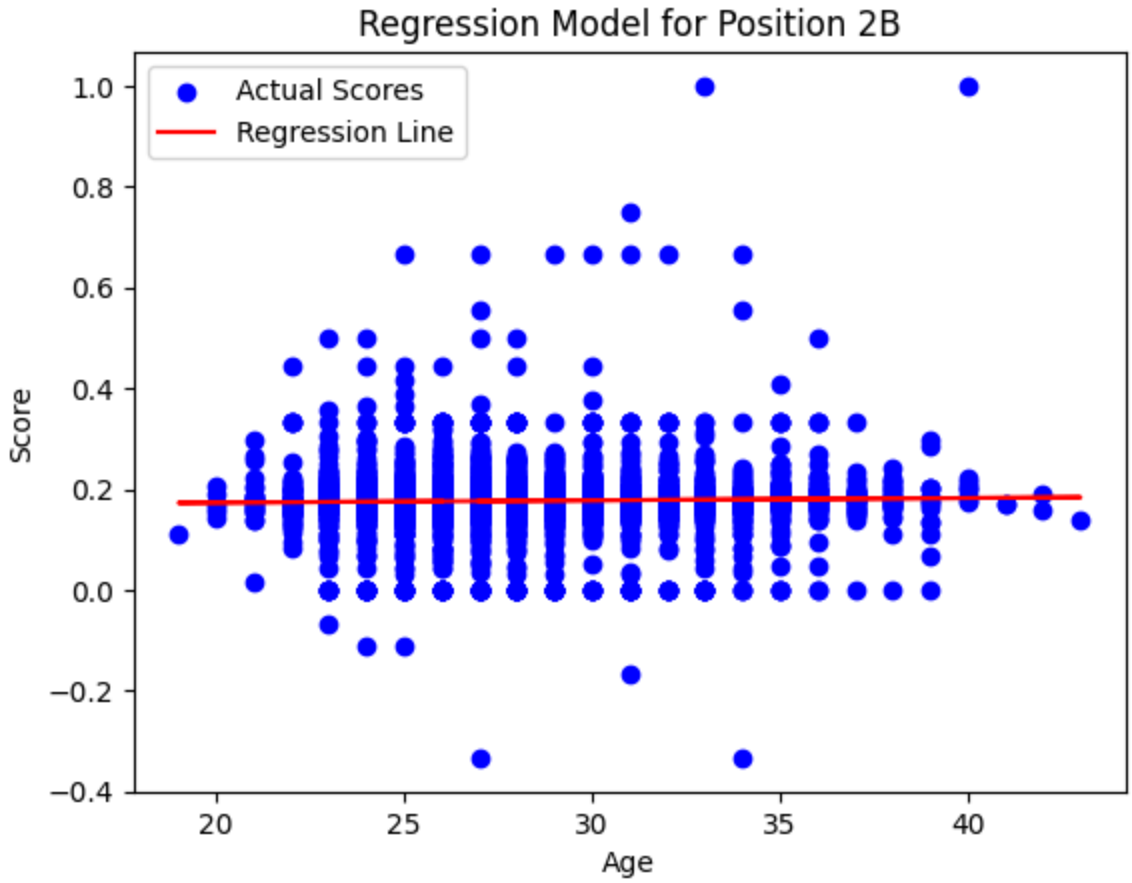
In [39]:
```python
import matplotlib.pyplot as plt

# Define a function to plot the regression line
def plot_regression_line(model, X, y, position):
    # Scatter plot of actual scores against age
    plt.scatter(X, y, color='blue', label='Actual Scores')
```

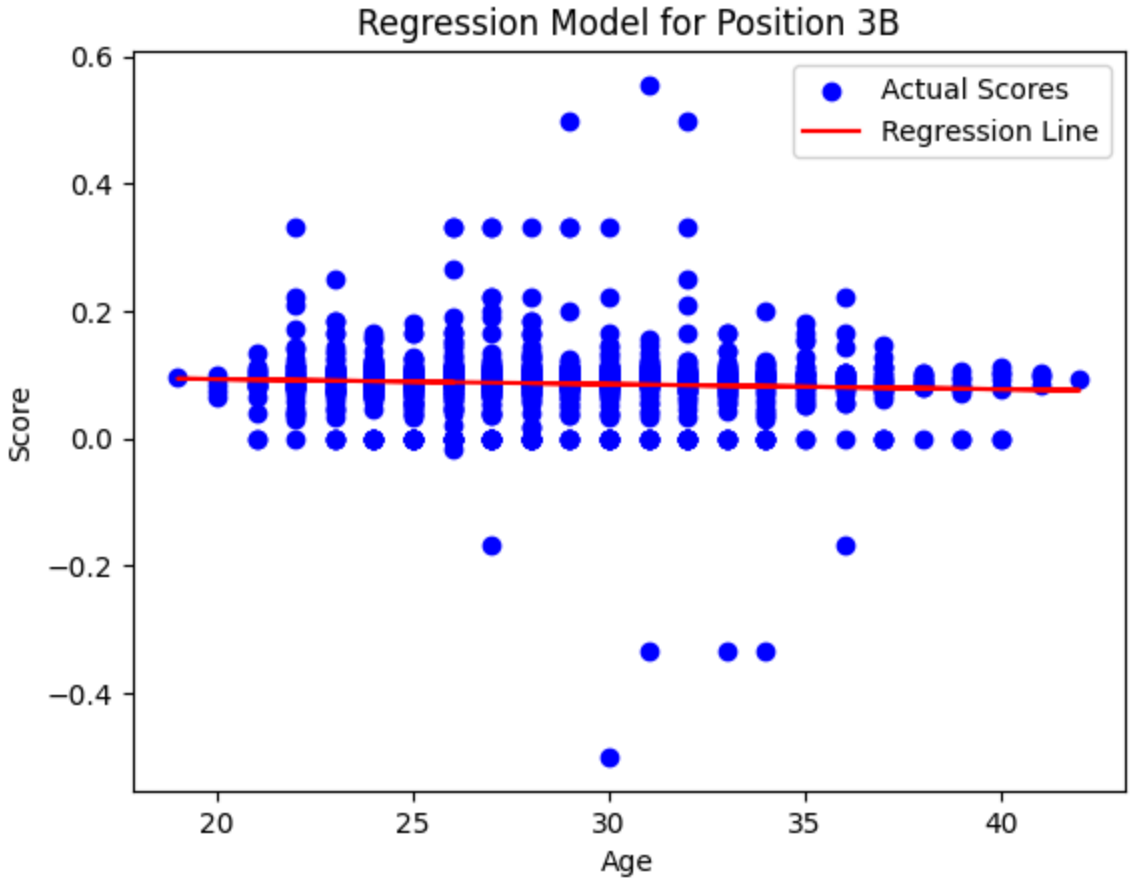
```
    # Plot the regression line
    plt.plot(X, model.predict(X), color='red', label='Regression Line')

    # Set labels and title
    plt.xlabel('Age')
    plt.ylabel('Score')
    plt.title(f'Regression Model for Position {position}')

    # Show legend
    plt.legend()

    # Show plot
    plt.show()

# Plot regression models for each position
for position, data in zip(positions, [c_data, p_data, b1_data, b2_data, b3_d
    # Make a copy of the data to avoid modifying the original DataFrame
    data_copy = data.copy()

    # Drop rows with missing values
    data_copy.dropna(subset=['score', 'age'], inplace=True)

    # Extract features (X) and target variable (y)
    X = data_copy[['age']]  # Assuming 'age' is the feature
    y = data_copy['score']  # Assuming 'score' is the target variable

    # Plot regression line
    plot_regression_line(models[position], X, y, position)
```

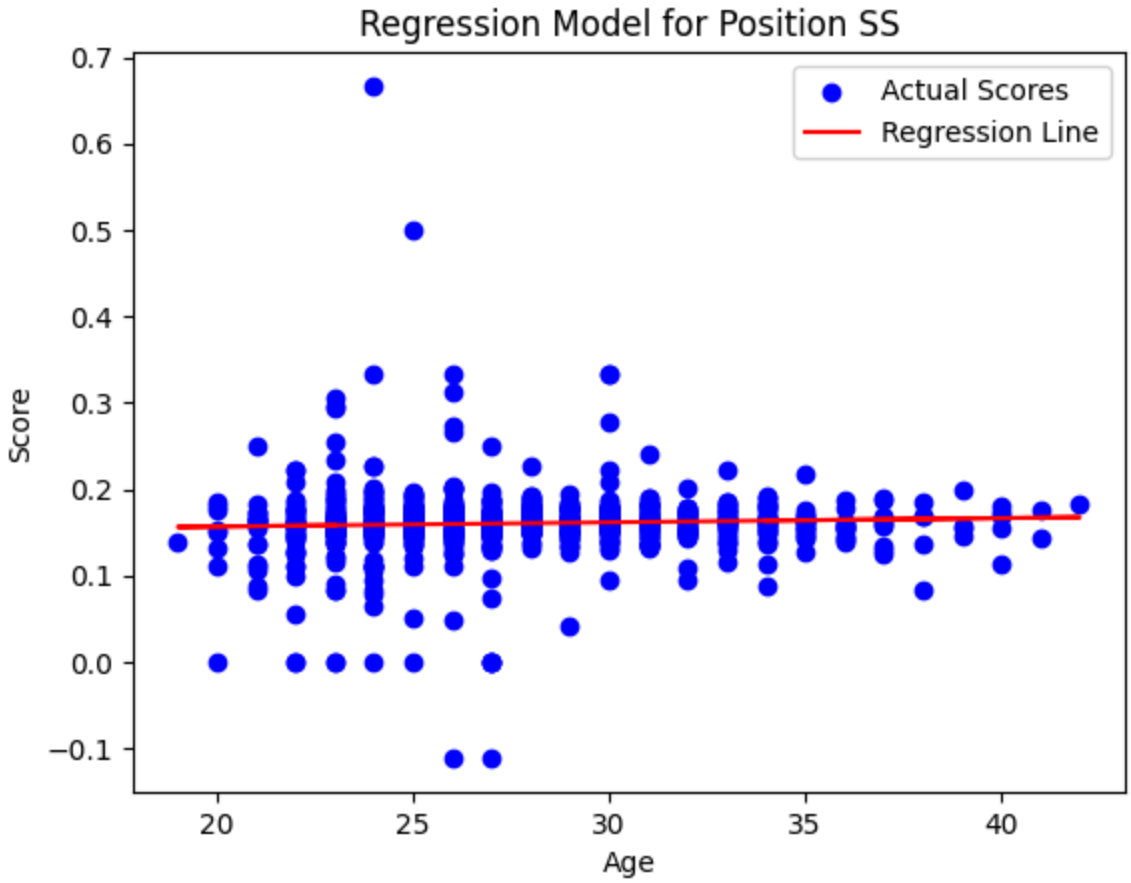

Regression Model for Position C
Actual Scores
Regression Line
Score
Age
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
20
25
30
35
40
45

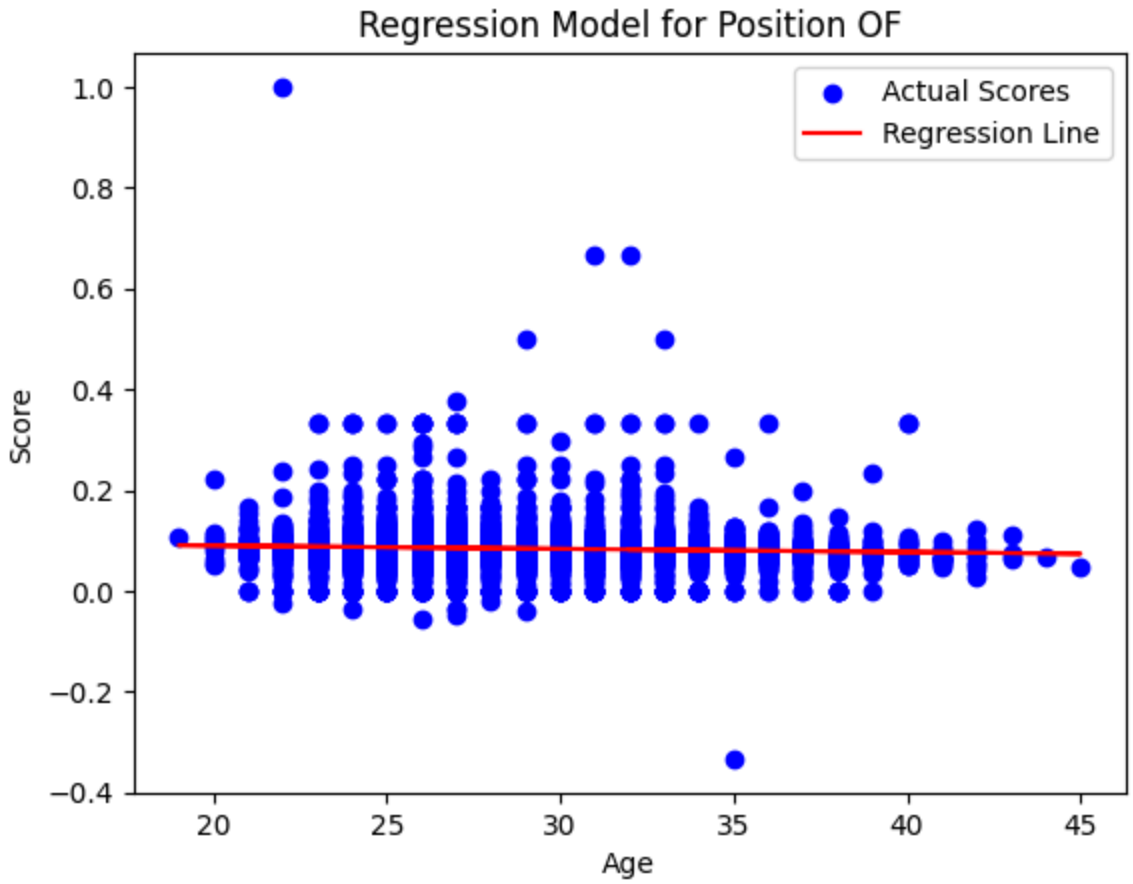

Regression Model for Position P
Actual Scores
Regression Line
Score
Age
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
20
25
30
35
40
45
50


Regression Model for Position 1B
Actual Scores
Regression Line
Score
Age
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
20
25
30
35
40
45
50


Regression Model for Position 2B
Actual Scores
Regression Line
Score
Age
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
20
25
30
35
40


Regression Model for Position 3B
Actual Scores
Regression Line
Score
Age
0.6
0.4
0.2
0.0
−0.2
−0.4
20
25
30
35
40


Regression Model for Position SS
Actual Scores
Regression Line
Score
Age
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
20
25
30
35
40

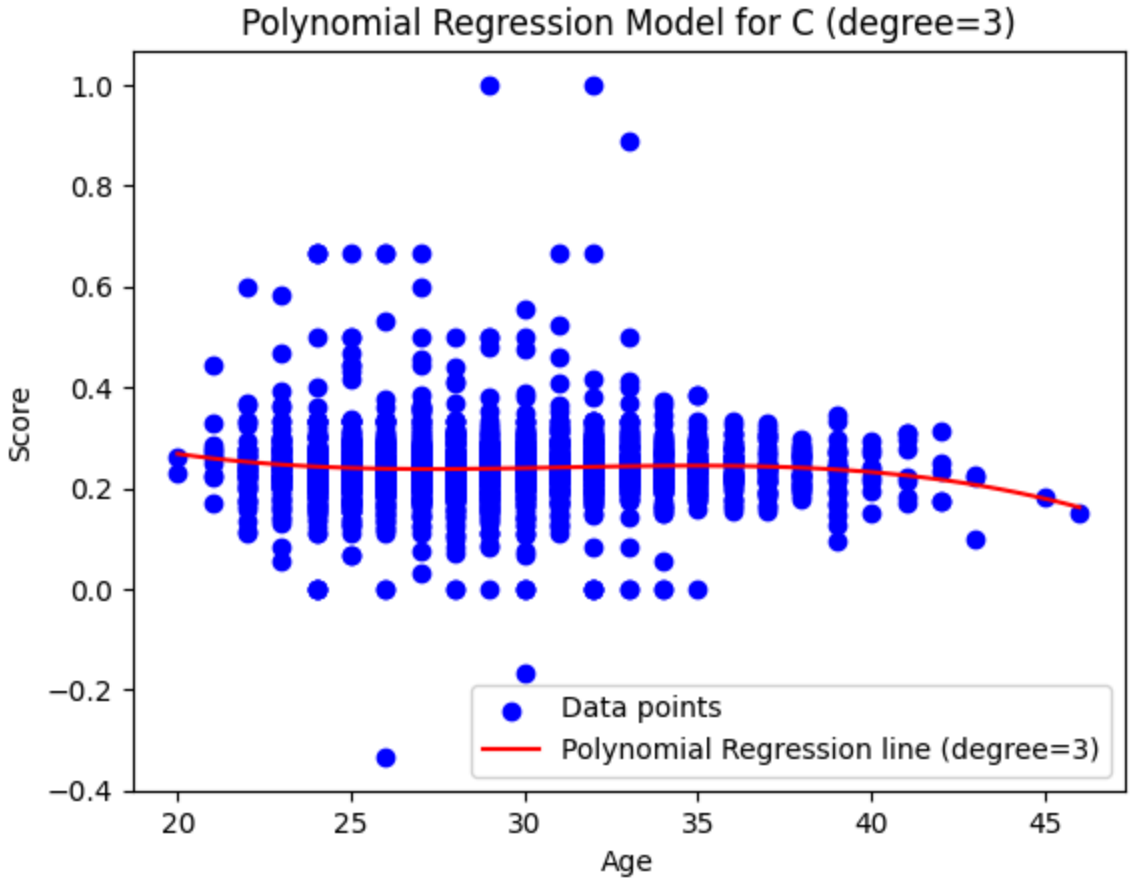


In [40]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Define a function to create and evaluate a polynomial regression model for
def create_and_evaluate_polynomial_regression_model(data, position, degree=3
    # Make a copy of the data to avoid modifying the original DataFrame
    data_copy = data.copy()

    # Drop rows with missing values
    data_copy.dropna(subset=['score', 'age'], inplace=True)

    # Extract features (X) and target variable (y)
    X = data_copy[['age']]  #  'age' is the feature
    y = data_copy['score']  # 'score' is the target variable

    # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

    # Create polynomial features
    polynomial_features = PolynomialFeatures(degree=degree)
    X_poly = polynomial_features.fit_transform(X_train)

    # Create a Linear Regression model
    model = LinearRegression()
```

```
    # Fit the model to the polynomial features
    model.fit(X_poly, y_train)

    # Evaluate the model on the testing data
    X_test_poly = polynomial_features.transform(X_test)
    y_pred = model.predict(X_test_poly)
    mse = mean_squared_error(y_test, y_pred)

    # Print model coefficients and evaluation metric
    print(f"Polynomial regression coefficients for {position} (degree={degre
    print("Intercept:", model.intercept_)
    print("Coefficients:", model.coef_)
    print("Mean Squared Error:", mse)

    # Visualize the regression model
    x_values = np.linspace(min(X['age']), max(X['age']), 100).reshape(-1, 1)
    x_values_poly = polynomial_features.transform(x_values)
    y_pred_poly = model.predict(x_values_poly)

    plt.scatter(X, y, color='blue', label='Data points')
    plt.plot(x_values, y_pred_poly, color='red', label='Polynomial Regressic
    plt.xlabel('Age')
    plt.ylabel('Score')
    plt.title(f'Polynomial Regression Model for {position} (degree={degree})
    plt.legend()
    plt.show()

    # Return the trained model
    return model

# Create and evaluate polynomial regression models for each position
positions = ['C', 'P', '1B', '2B', '3B', 'SS', 'OF']

for position, data in zip(positions, [c_data, p_data, b1_data, b2_data, b3_d
    print(f"Position: {position}")
    create_and_evaluate_polynomial_regression_model(data, position)
    print("\n")
```

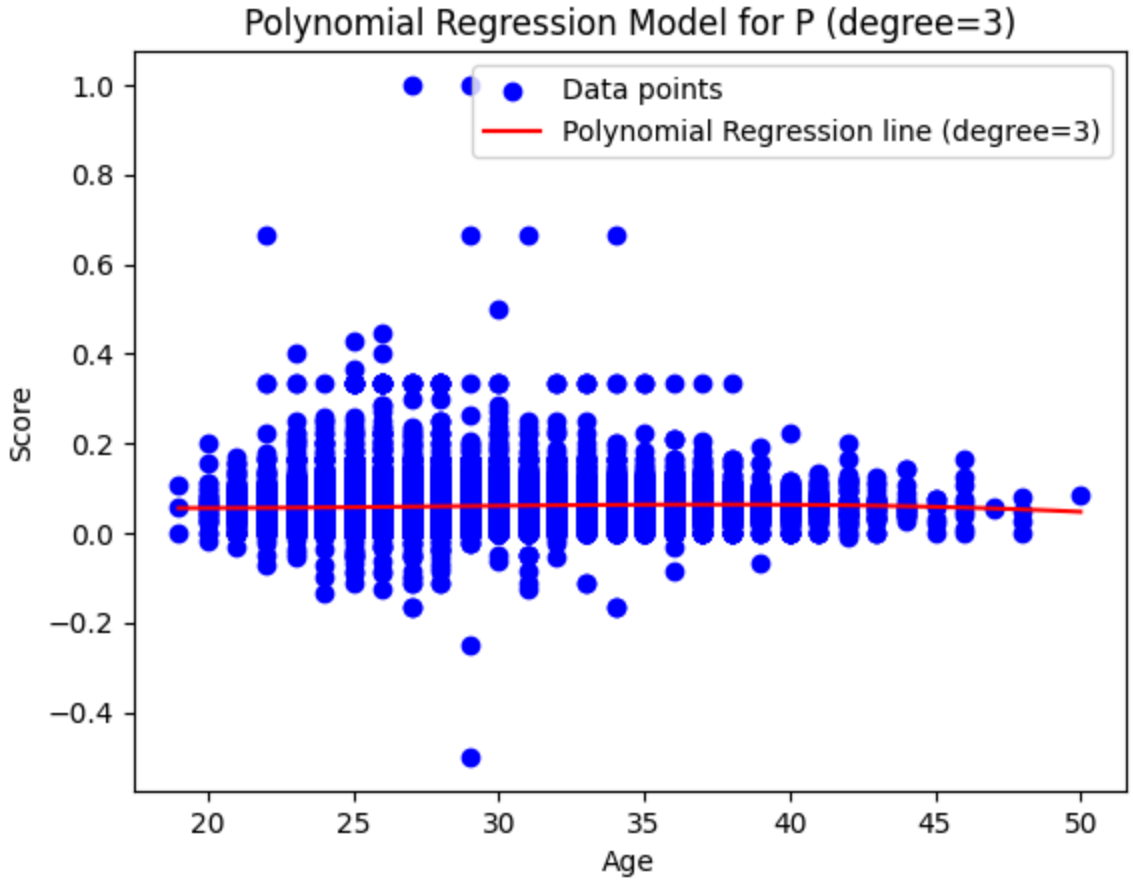
```
Position: C
Polynomial regression coefficients for C (degree=3):
Intercept: 1.1065441186903677
Coefficients: [ 0.00000000e+00 -8.60333022e-02  2.80601012e-03 -3.00491807e-
05]
Mean Squared Error: 0.004515706712620159
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
```



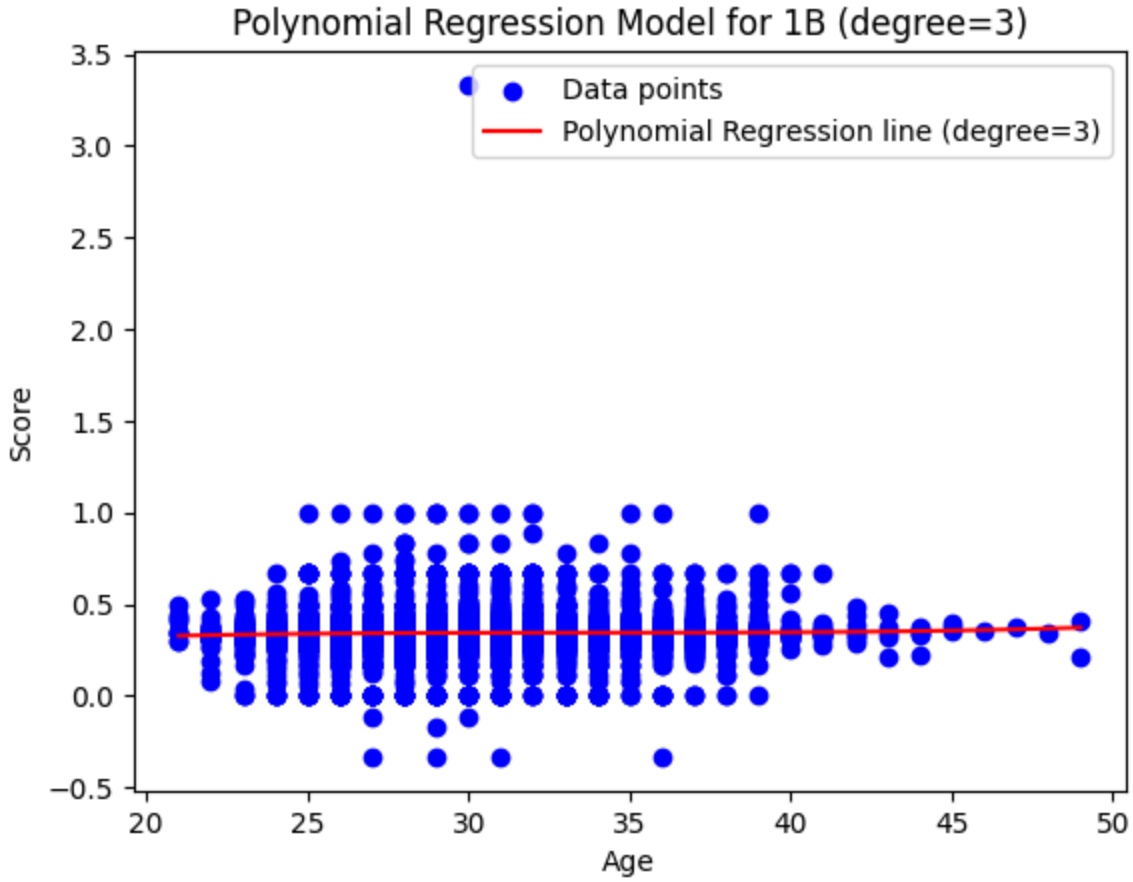

```
Position: P
Polynomial regression coefficients for P (degree=3):
Intercept: 0.09868130777480902
Coefficients: [ 0.00000000e+00 -5.47587929e-03  2.16887760e-04 -2.56101910e-
06]
Mean Squared Error: 0.0017192417027446537
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
```



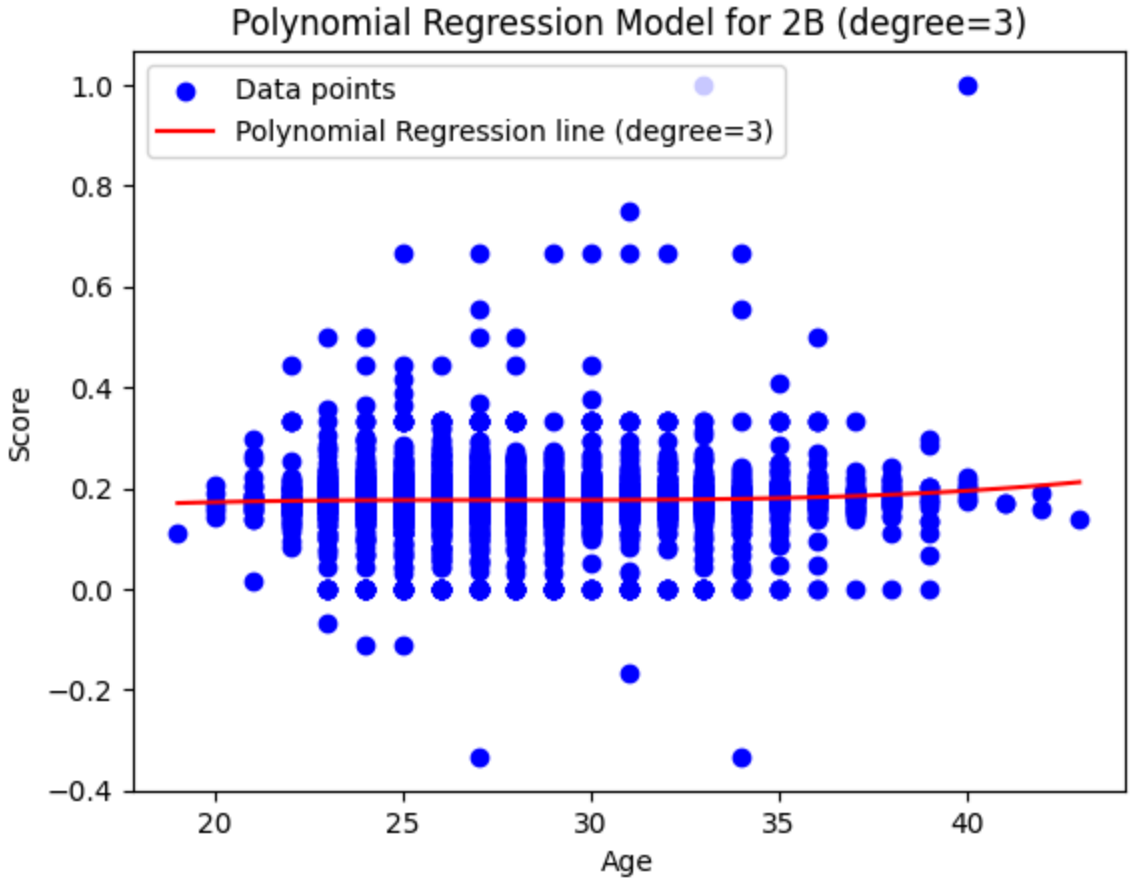

```
Position: 1B
Polynomial regression coefficients for 1B (degree=3):
Intercept: 0.05383902251302192
Coefficients: [ 0.00000000e+00  2.55734482e-02 -7.53897410e-04  7.43728477e-
06]
Mean Squared Error: 0.012262566356556199
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
```



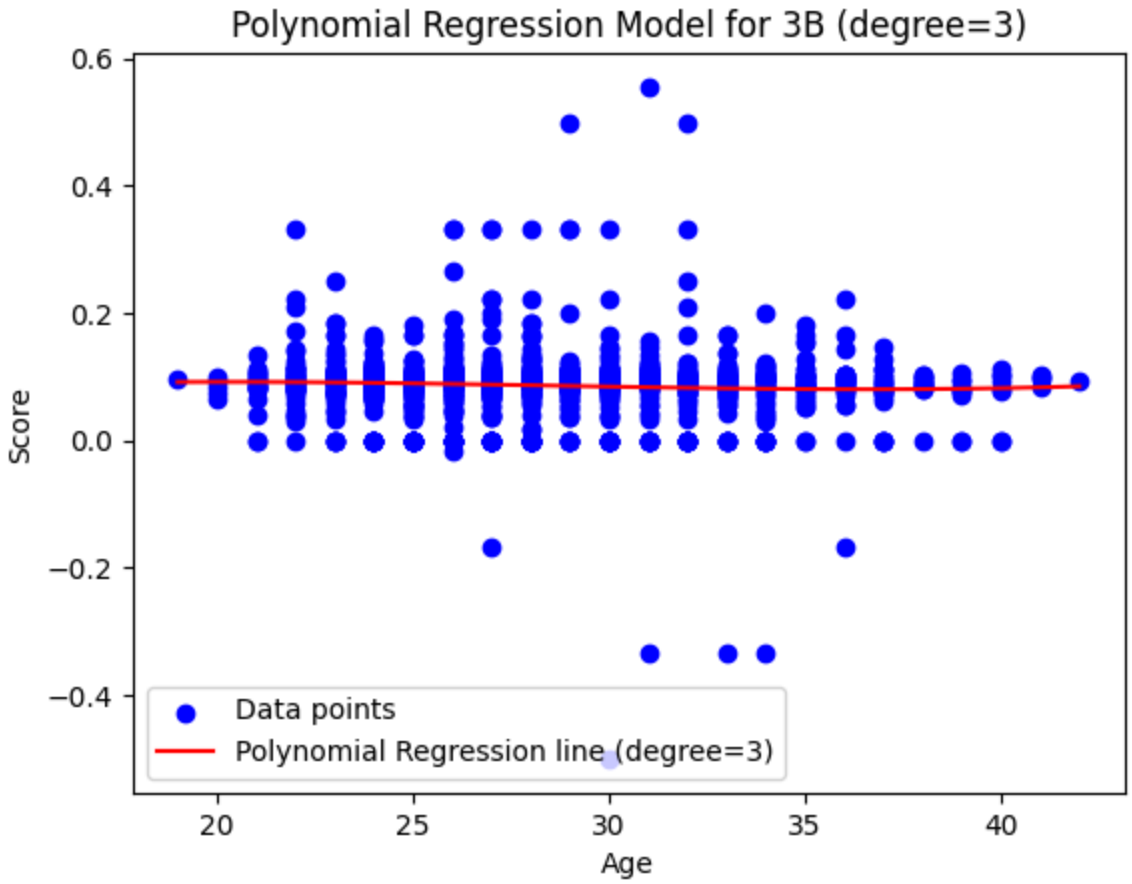

```
Position: 2B
Polynomial regression coefficients for 2B (degree=3):
Intercept: -0.03182748067711802
Coefficients: [ 0.00000000e+00  2.26456922e-02 -8.20496162e-04  9.89753763e-
06]
Mean Squared Error: 0.0034813626580982
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
```



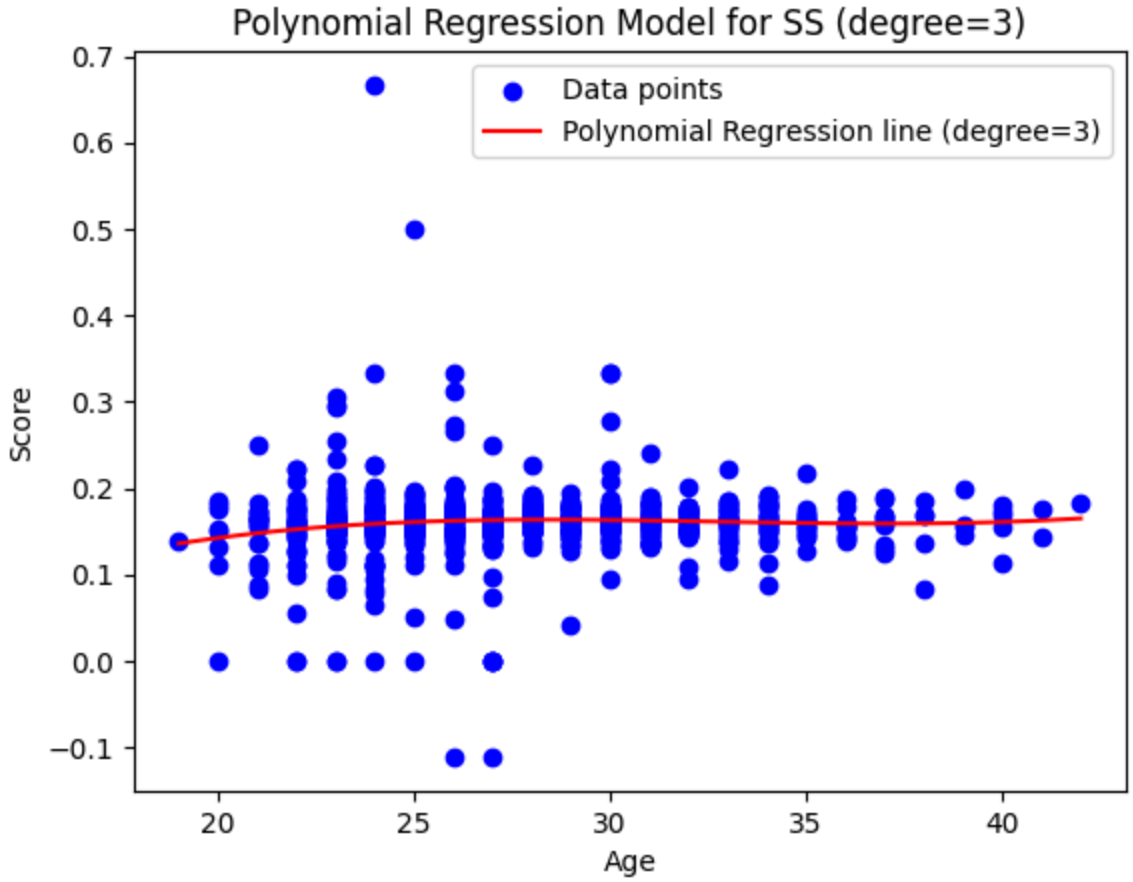

```
Position: 3B
Polynomial regression coefficients for 3B (degree=3):
Intercept: -0.007225942102750599
Coefficients: [ 0.00000000e+00  1.20001958e-02 -4.59212461e-04  5.37870205e-
06]
Mean Squared Error: 0.0033737129370893008
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
```



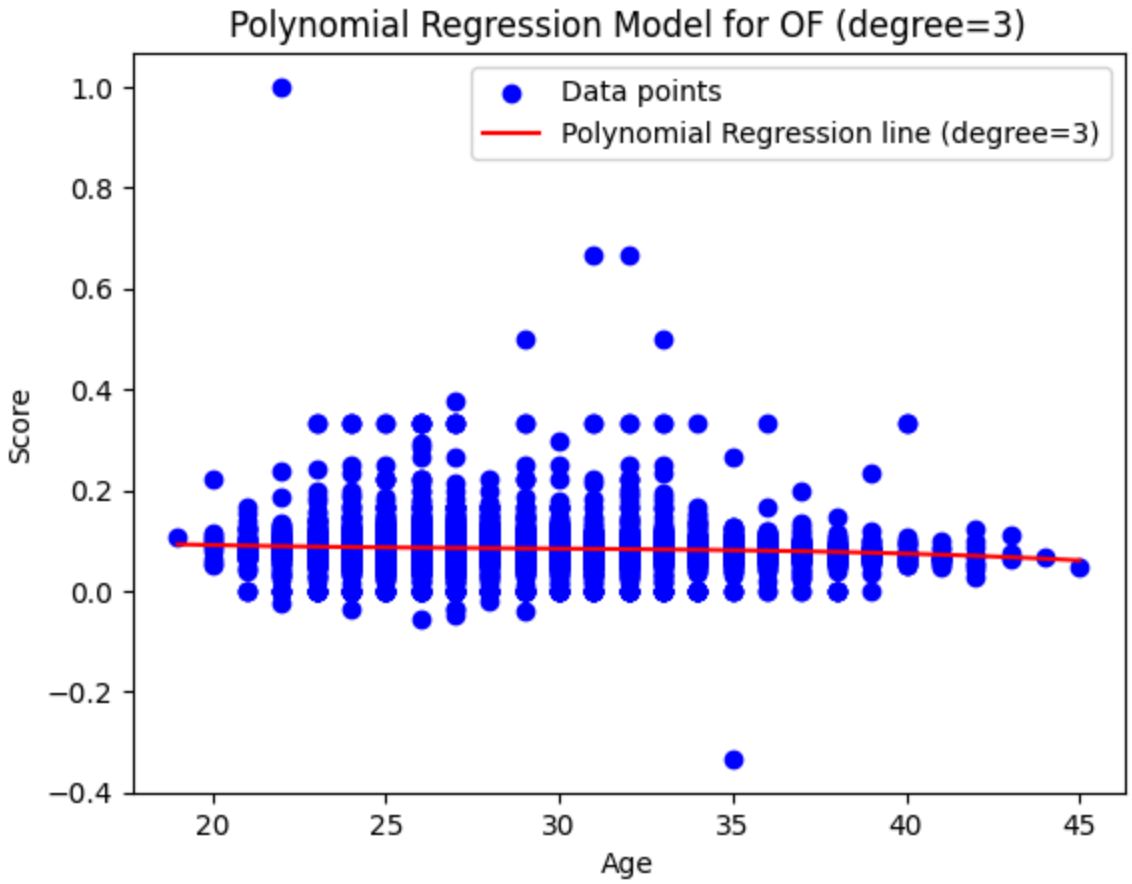

```
Position: SS
Polynomial regression coefficients for SS (degree=3):
Intercept: -0.30124882663500696
Coefficients: [ 0.00000000e+00  4.39143998e-02 -1.36468816e-03  1.38897324e-
05]
Mean Squared Error: 0.000953057116900442
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
```




```
Position: OF
Polynomial regression coefficients for OF (degree=3):
Intercept: 0.19097983680206174
Coefficients: [ 0.00000000e+00 -1.02148285e-02  3.39062491e-04 -3.92259797e-
06]
Mean Squared Error: 0.0014016215173596443
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
```



In [41]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Function to handle missing values by imputing with the mean
def handle_missing_values(data):
    # Check for missing values in the 'score' column
    missing_values = data['score'].isnull().sum()
    print("Number of missing values in 'score' column:", missing_values)

    # Impute missing values with the mean
    mean_score = data['score'].mean()
    data['score'].fillna(mean_score, inplace=True)

# Accumulate polynomial features and predictions for each position
X_poly_all = []
y_pred_all = []

# Define a function to create and evaluate a polynomial regression model for
def create_and_evaluate_polynomial_regression_model(data, position, degree=2
    # Handle missing values
    handle_missing_values(data)

    # Extract features (X) and target variable (y)
```

```
    X = data[['age']]  # 'age' is the feature
    y = data['score']  # 'score' is the target variable

    # Create polynomial features
    polynomial_features = PolynomialFeatures(degree=degree)
    X_poly = polynomial_features.fit_transform(X)

    # Create a Linear Regression model
    model = LinearRegression()

    # Fit the model to the data
    model.fit(X_poly, y)

    # Append polynomial features and predictions to the lists
    X_poly_all.append(X_poly)
    y_pred_all.append(model.predict(X_poly))

# Create and evaluate polynomial regression models for each position
positions = ['C', 'P', '1B', '2B', '3B', 'SS', 'OF']

for position, data in zip(positions, [c_data, p_data, b1_data, b2_data, b3_d
    print(f"Position: {position}")
    create_and_evaluate_polynomial_regression_model(data, position, degree=3
    print("\n")

# Plot all the curved regression lines on the same plot
plt.figure(figsize=(10, 6))
plt.scatter(c_data['age'], c_data['score'], color='blue', label='C')
plt.scatter(p_data['age'], p_data['score'], color='red', label='P')
plt.scatter(b1_data['age'], b1_data['score'], color='green', label='1B')
plt.scatter(b2_data['age'], b2_data['score'], color='orange', label='2B')
plt.scatter(b3_data['age'], b3_data['score'], color='purple', label='3B')
plt.scatter(ss_data['age'], ss_data['score'], color='yellow', label='SS')
plt.scatter(of_data['age'], of_data['score'], color='black', label='OF')

# Plot the curved regression lines
for i in range(len(positions)):
    plt.plot(X_poly_all[i][:, 1], y_pred_all[i], label=f'{positions[i]}', li

plt.xlabel('Age')
plt.ylabel('Score')
plt.title('Curved Regression Models for Different Positions')
plt.legend()
plt.grid(True)
plt.show()
```

```
Position: C
Number of missing values in 'score' column: 116


Position: P
Number of missing values in 'score' column: 850
```

```
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/
stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['score'].fillna(mean_score, inplace=True)
```

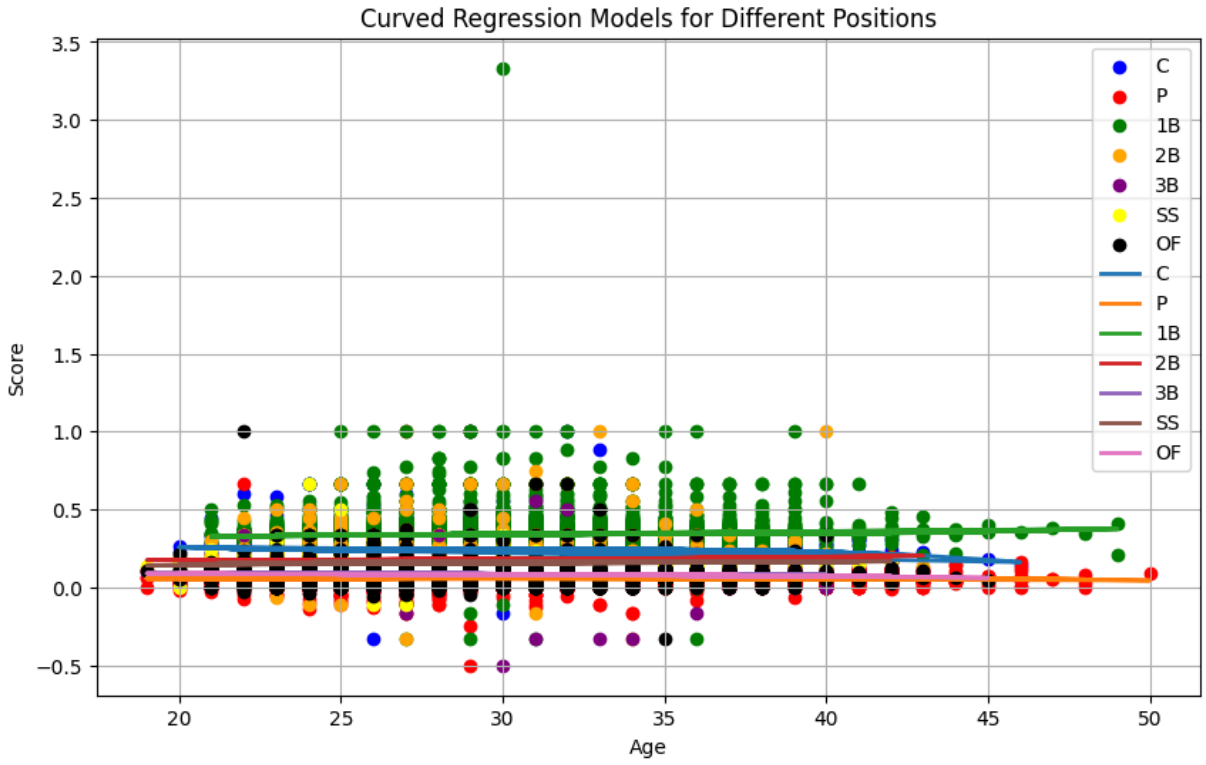
```
Position: 1B
Number of missing values in 'score' column: 173


Position: 2B
Number of missing values in 'score' column: 119


Position: 3B
Number of missing values in 'score' column: 50
```

```
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/
stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/
stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/
stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['score'].fillna(mean_score, inplace=True)
```

```
Position: SS
Number of missing values in 'score' column: 30


Position: OF
Number of missing values in 'score' column: 421
```

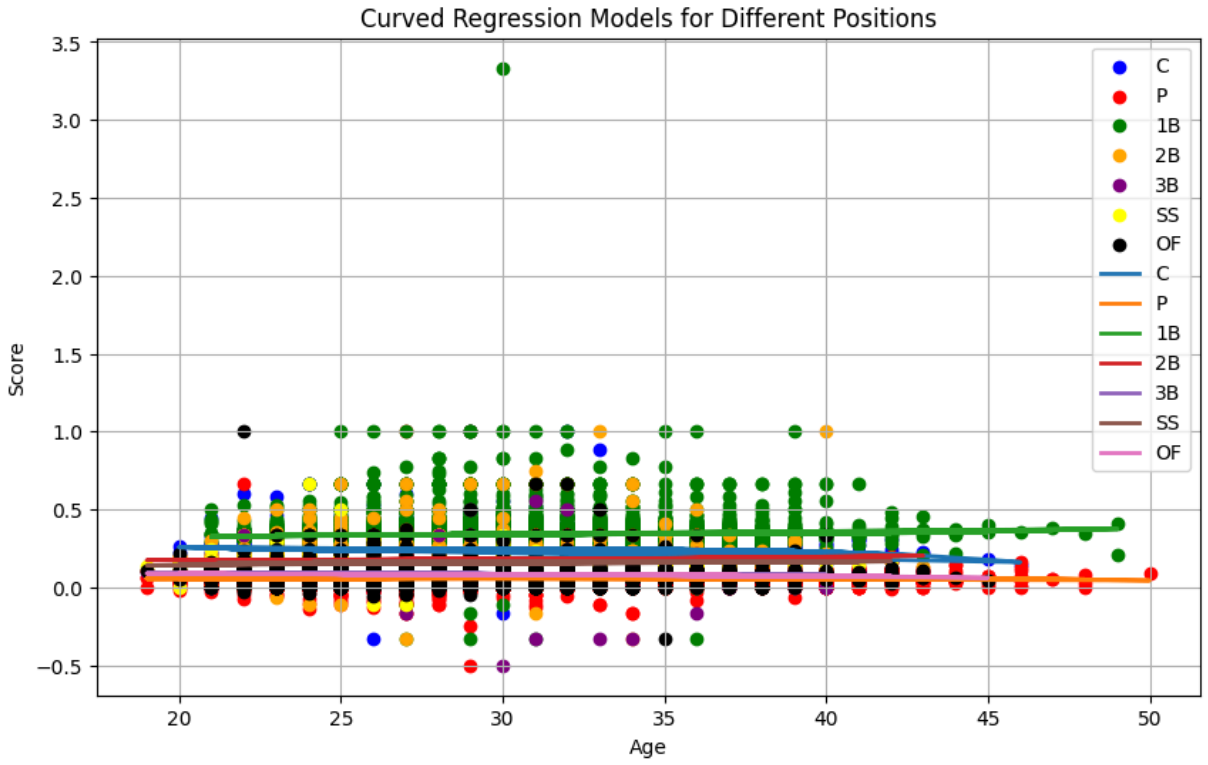
```
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/
stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  data['score'].fillna(mean_score, inplace=True)
/var/folders/_s/0rsc9b4j70z86mpsqkw4l83r0000gn/T/ipykernel_4062/2211538233.p
y:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behave
s as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'd
f.method({col: value}, inplace=True)' or df[col] = df[col].method(value) ins
tead, to perform the operation inplace on the original object.


  data['score'].fillna(mean_score, inplace=True)
```



In [42]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Accumulate polynomial features and predictions for each position
X_poly_all = []
y_pred_all = []

# Define a function to create and evaluate a polynomial regression model for
def create_and_evaluate_polynomial_regression_model(data, position, degree=2
    # Extract features (X) and target variable (y)
    X = data[['age']]  # 'age' is the feature
    y = data['score']  #  'score' is the target variable

    # Create polynomial features
    polynomial_features = PolynomialFeatures(degree=degree)
    X_poly = polynomial_features.fit_transform(X)

    # Create a Linear Regression model
    model = LinearRegression()

    # Fit the model to the data
    model.fit(X_poly, y)

    # Append polynomial features and predictions to the lists
    X_poly_all.append(X_poly)
    y_pred_all.append(model.predict(X_poly))

# Create and evaluate polynomial regression models for each position
```

```
positions = ['C', 'P', '1B', '2B', '3B', 'SS', 'OF']

for position, data in zip(positions, [c_data, p_data, b1_data, b2_data, b3_d
    print(f"Position: {position}")
    create_and_evaluate_polynomial_regression_model(data, position, degree=3
    print("\n")

# Plot all the curved regression lines on the same plot
plt.figure(figsize=(10, 6))
plt.scatter(c_data['age'], c_data['score'], color='blue', label='C')
plt.scatter(p_data['age'], p_data['score'], color='red', label='P')
plt.scatter(b1_data['age'], b1_data['score'], color='green', label='1B')
plt.scatter(b2_data['age'], b2_data['score'], color='orange', label='2B')
plt.scatter(b3_data['age'], b3_data['score'], color='purple', label='3B')
plt.scatter(ss_data['age'], ss_data['score'], color='yellow', label='SS')
plt.scatter(of_data['age'], of_data['score'], color='black', label='OF')

# Plot the curved regression lines
for i in range(len(positions)):
    plt.plot(X_poly_all[i][:, 1], y_pred_all[i], label=f'{positions[i]}', li

plt.xlabel('Age')
plt.ylabel('Score')
plt.title('Curved Regression Models for Different Positions')
plt.legend()
plt.grid(True)
plt.show()
```

```
Position: C


Position: P


Position: 1B


Position: 2B


Position: 3B


Position: SS


Position: OF
```

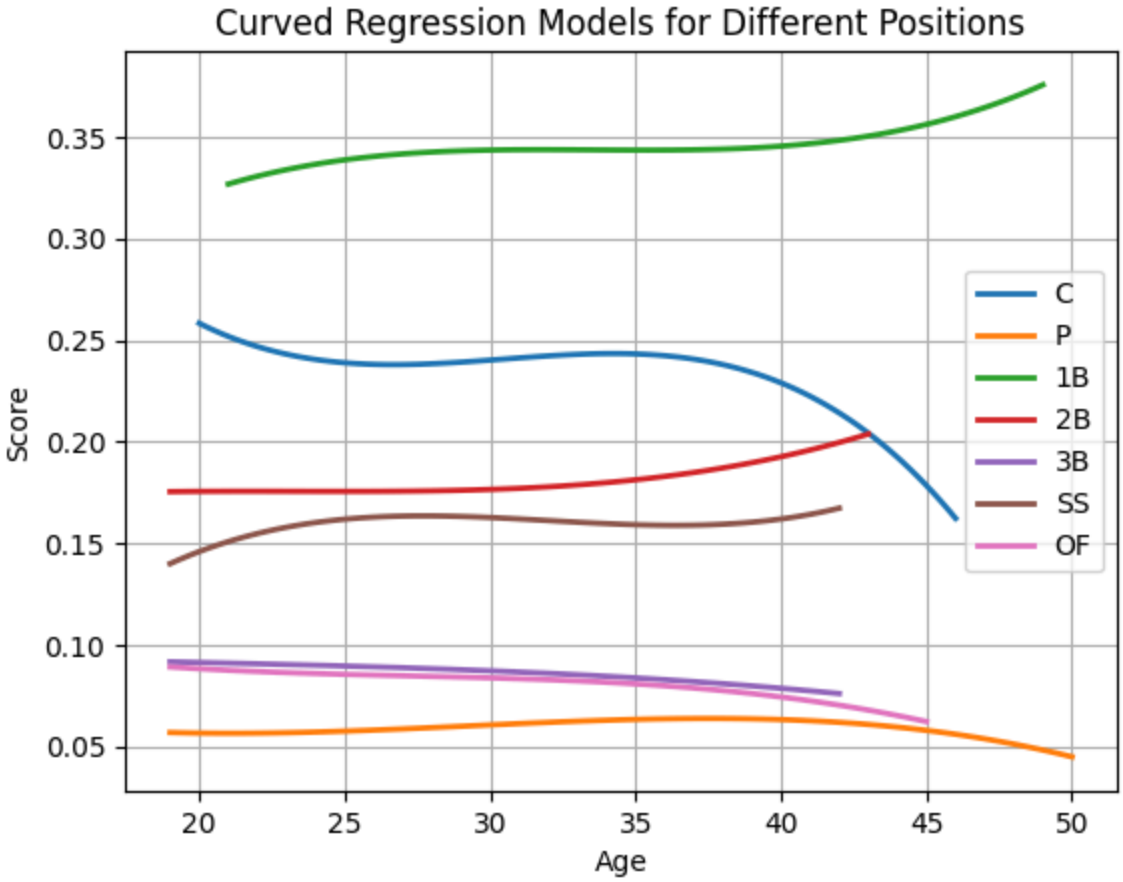


In [ ]:

In [43]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Define a function to create and evaluate a polynomial regression model for
def create_and_evaluate_polynomial_regression_model(data, position, degree=2
    # Extract features (X) and target variable (y)
    X = data[['age']]  #  'age' is the feature
    y = data['score']  #  'score' is the target variable

    # Create polynomial features
    polynomial_features = PolynomialFeatures(degree=degree)
    X_poly = polynomial_features.fit_transform(X)

    # Create a Linear Regression model
    model = LinearRegression()

    # Fit the model to the data
    model.fit(X_poly, y)

    # Plot the curved regression line
    x_values = np.linspace(min(X['age']), max(X['age']), 100).reshape(-1, 1)
    x_poly = polynomial_features.transform(x_values)
    y_pred = model.predict(x_poly)
    plt.plot(x_values, y_pred, label=f'{position}', linewidth=2)

# Create and evaluate polynomial regression models for each position
positions = ['C', 'P', '1B', '2B', '3B', 'SS', 'OF']
```

```
for position, data in zip(positions, [c_data, p_data, b1_data, b2_data, b3_d
    create_and_evaluate_polynomial_regression_model(data, position, degree=3

plt.xlabel('Age')
plt.ylabel('Score')
plt.title('Curved Regression Models for Different Positions')
plt.legend()
plt.grid(True)
plt.show()
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/base.py:493: UserWarning: X does not have valid feature names, but Pol
ynomialFeatures was fitted with feature names
  warnings.warn(
```



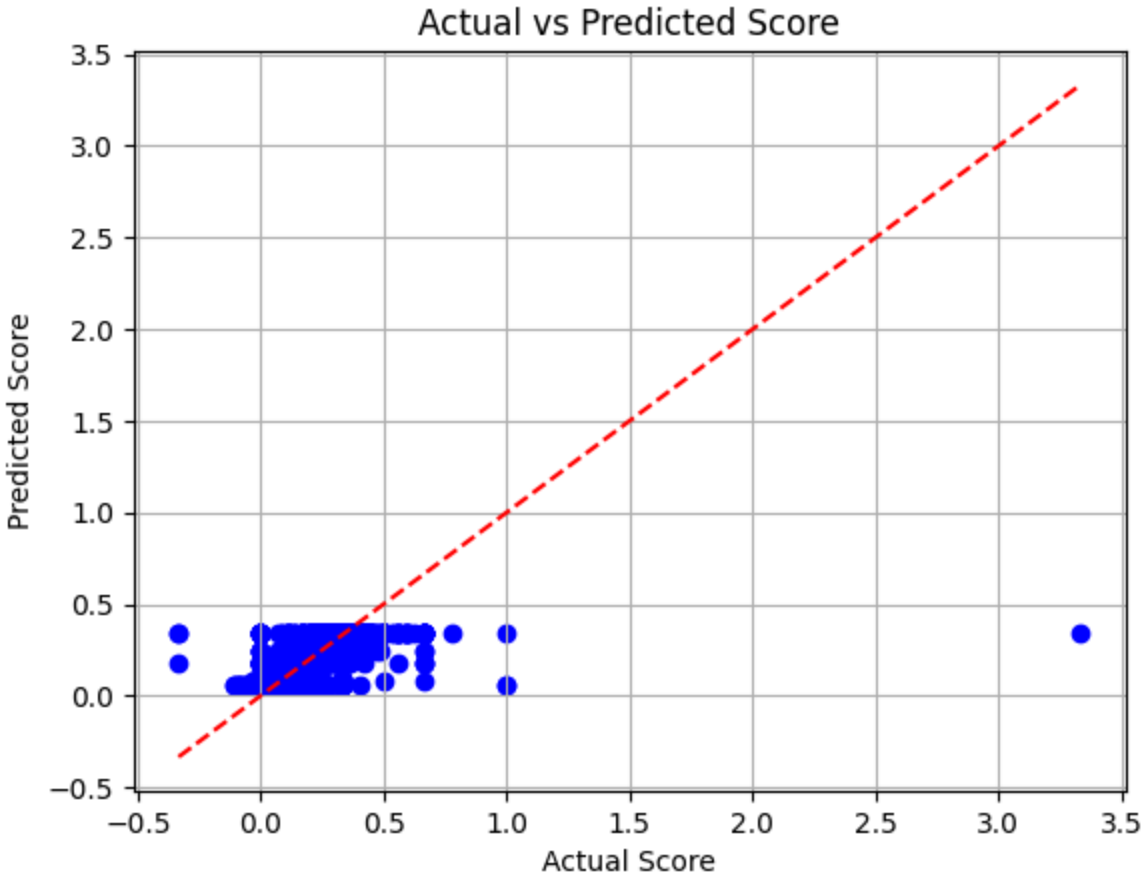
In [44]:
```
import pandas as pd

# 1. Change all positions in the 'OF' table to 'OF'
of_data['position'] = 'OF'
c_data.rename(columns={'pos': 'position'}, inplace=True)

# 2. Combine all tables into one big table with only the 'age', 'position',
combined_data = pd.concat([c_data[['age', 'position', 'score']],
                           p_data[['age', 'position', 'score']],
                           b1_data[['age', 'position', 'score']],
                           b2_data[['age', 'position', 'score']],
                           b3_data[['age', 'position', 'score']],
                           ss_data[['age', 'position', 'score']],
                           of_data[['age', 'position', 'score']]])

# 3. Create a multiple regression model using 'age' and 'position' as input
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error

# Split the data into features (X) and target variable (y)
X = combined_data[['age', 'position']]
y = combined_data['score']

# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

# Define preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(), ['position'])
    ],
    remainder='passthrough'
)

# Create a pipeline with preprocessing and linear regression model
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# Fit the model to the training data
model.fit(X_train, y_train)

# Evaluate the model on the testing data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

# Print model coefficients and evaluation metric
print("Model Coefficients:")
print(model.named_steps['regressor'].coef_)
print("Intercept:", model.named_steps['regressor'].intercept_)
print("Mean Squared Error:", mse)
```

```
Model Coefficients:
[ 0.17636603  0.01258472 -0.07750667  0.07651909 -0.08024597 -0.10487681
 -0.00284039  0.00023343]
Intercept: 0.15764239107702585
Mean Squared Error: 0.00450618246162656
```

In [45]:

```
import matplotlib.pyplot as plt
import numpy as np

# Visualize the model predictions
plt.scatter(y_test, y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='
plt.xlabel('Actual Score')
plt.ylabel('Predicted Score')
plt.title('Actual vs Predicted Score')
plt.grid(True)
plt.show()

# Calculate R-squared
from sklearn.metrics import r2_score
r_squared = r2_score(y_test, y_pred)
print("R-squared:", r_squared)
```



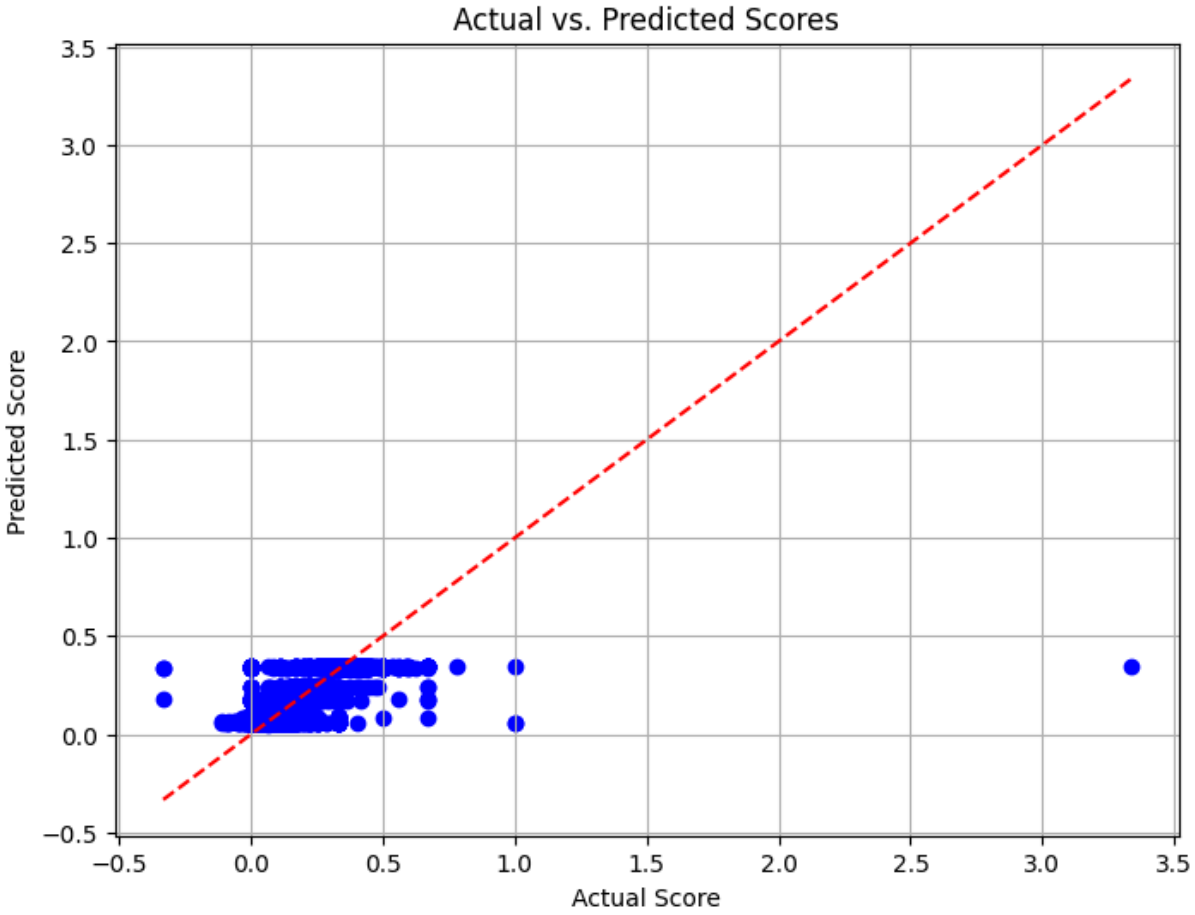

```
R-squared: 0.6882909653134626
```

In [46]:
```
import pandas as pd

# 1. Change all positions in the 'OF' table to 'OF'
of_data['position'] = 'OF'
c_data.rename(columns={'pos': 'position'}, inplace=True)

# 2. Combine all tables into one big table with only the 'age', 'position',
combined_data = pd.concat([c_data[['age', 'position', 'score']],
                           p_data[['age', 'position', 'score']],
                           b1_data[['age', 'position', 'score']],
                           b2_data[['age', 'position', 'score']],
                           b3_data[['age', 'position', 'score']],
                           ss_data[['age', 'position', 'score']],
                           of_data[['age', 'position', 'score']]])

# 3. Add 'height' and 'weight' columns from joined_data_copy to the combined
combined_data['height'] = joined_data_copy['height']
combined_data['weight'] = joined_data_copy['weight']

# 4. Create a multiple regression model using 'age', 'position', 'height', a
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error
```

```
# Split the data into features (X) and target variable (y)
X = combined_data[['age', 'position', 'height', 'weight']]
y = combined_data['score']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

# Define preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(), ['position'])
    ],
    remainder='passthrough'
)

# Create a pipeline with preprocessing and linear regression model
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# Fit the model to the training data
model.fit(X_train, y_train)

# Evaluate the model on the testing data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

# Print model coefficients and evaluation metric
print("Model Coefficients:")
print(model.named_steps['regressor'].coef_)
print("Intercept:", model.named_steps['regressor'].intercept_)
print("Mean Squared Error:", mse)
```

```
Model Coefficients:
[-4.30930173e+10 -4.30930173e+10 -4.30930173e+10 -4.30930173e+10
 -4.30930173e+10 -4.30930173e+10 -4.30930173e+10  1.84989365e-04
 -1.44137179e-04 -6.64031894e-05]
Intercept: 43093017349.75238
Mean Squared Error: 0.004497841085920721
```

In [47]:
```
import matplotlib.pyplot as plt

# Visualize predicted vs. actual scores
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red'
plt.xlabel('Actual Score')
plt.ylabel('Predicted Score')
plt.title('Actual vs. Predicted Scores')
plt.grid(True)
plt.show()
# Evaluate the model and calculate R^2 score
r_squared = model.score(X_test, y_test)
print("R^2 Score:", r_squared)
```




```
R^2 Score: 0.6888679686175602
```

In [48]:
```
import pandas as pd

# 1. Change all positions in the 'OF' table to 'OF'
of_data['position'] = 'OF'
c_data.rename(columns={'pos': 'position'}, inplace=True)

# 2. Combine all tables into one big table with only the 'age', 'position',
combined_data = pd.concat([c_data[['age', 'position', 'score']],
                           p_data[['age', 'position', 'score']],
                           b1_data[['age', 'position', 'score']],
                           b2_data[['age', 'position', 'score']],
                           b3_data[['age', 'position', 'score']],
                           ss_data[['age', 'position', 'score']],
                           of_data[['age', 'position', 'score']]])

# 3. Add 'height', 'weight', and 'double_plays' columns from joined_data_cop
combined_data['height'] = joined_data_copy['height']
combined_data['weight'] = joined_data_copy['weight']
combined_data['double_plays'] = joined_data_copy['double_plays']

# 4. Create a multiple regression model using 'age', 'position', 'height', '
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
```

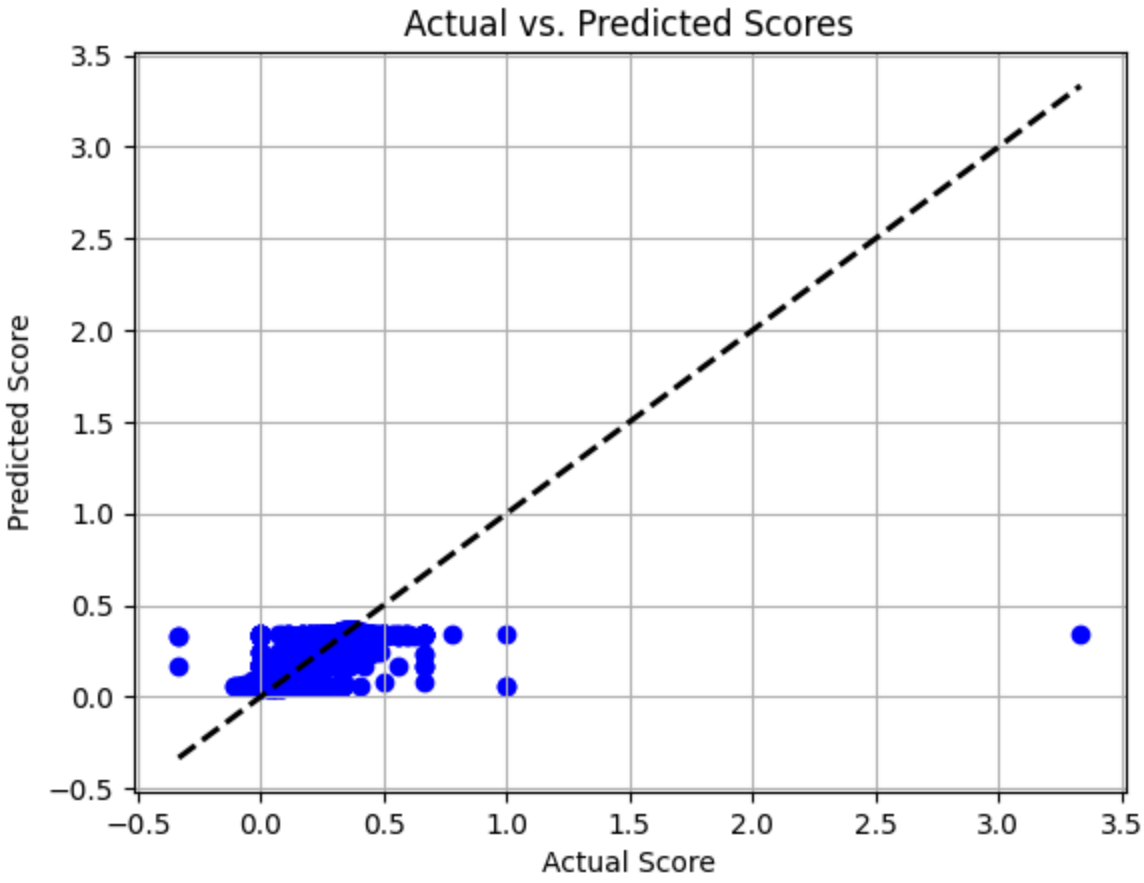
```
from sklearn.metrics import mean_squared_error

# Split the data into features (X) and target variable (y)
X = combined_data[['age', 'position', 'height', 'weight', 'double_plays']]
y = combined_data['score']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

# Define preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(), ['position'])
    ],
    remainder='passthrough'
)

# Create a pipeline with preprocessing and linear regression model
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# Fit the model to the training data
model.fit(X_train, y_train)

# Evaluate the model on the testing data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

# Print model coefficients and evaluation metric
print("Model Coefficients:")
print(model.named_steps['regressor'].coef_)
print("Intercept:", model.named_steps['regressor'].intercept_)
print("Mean Squared Error:", mse)
```

```
Model Coefficients:
[-4.32121119e+10 -4.32121119e+10 -4.32121119e+10 -4.32121119e+10
 -4.32121119e+10 -4.32121119e+10 -4.32121119e+10  1.80965209e-04
 -1.51298006e-04 -8.45247989e-05  1.76693582e-04]
Intercept: 43212111863.8702
Mean Squared Error: 0.004491026206914964
```

In [49]:

```
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score

# Visualize predicted vs. actual scores
plt.scatter(y_test, y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--',
plt.xlabel('Actual Score')
plt.ylabel('Predicted Score')
plt.title('Actual vs. Predicted Scores')
plt.grid(True)
plt.show()

# Calculate R^2 score
```

```
r2 = r2_score(y_test, y_pred)
print("R^2 Score:", r2)
```




```
R^2 Score: 0.6893393785914081
```

In [50]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error

# 1. Change all positions in the 'OF' table to 'OF'
of_data['position'] = 'OF'
c_data.rename(columns={'pos': 'position'}, inplace=True)
# 2. Combine all tables into one big table with only the 'age', 'position',
combined_data = pd.concat([c_data[['age', 'position', 'score']],
                           p_data[['age', 'position', 'score']],
                           b1_data[['age', 'position', 'score']],
                           b2_data[['age', 'position', 'score']],
                           b3_data[['age', 'position', 'score']],
                           ss_data[['age', 'position', 'score']],
                           of_data[['age', 'position', 'score']]])

# 3. Add 'height', 'weight', 'double_plays', 'throws', 'bats', and 'g' colum
combined_data['height'] = joined_data_copy['height']
combined_data['weight'] = joined_data_copy['weight']
combined_data['double_plays'] = joined_data_copy['double_plays']
```

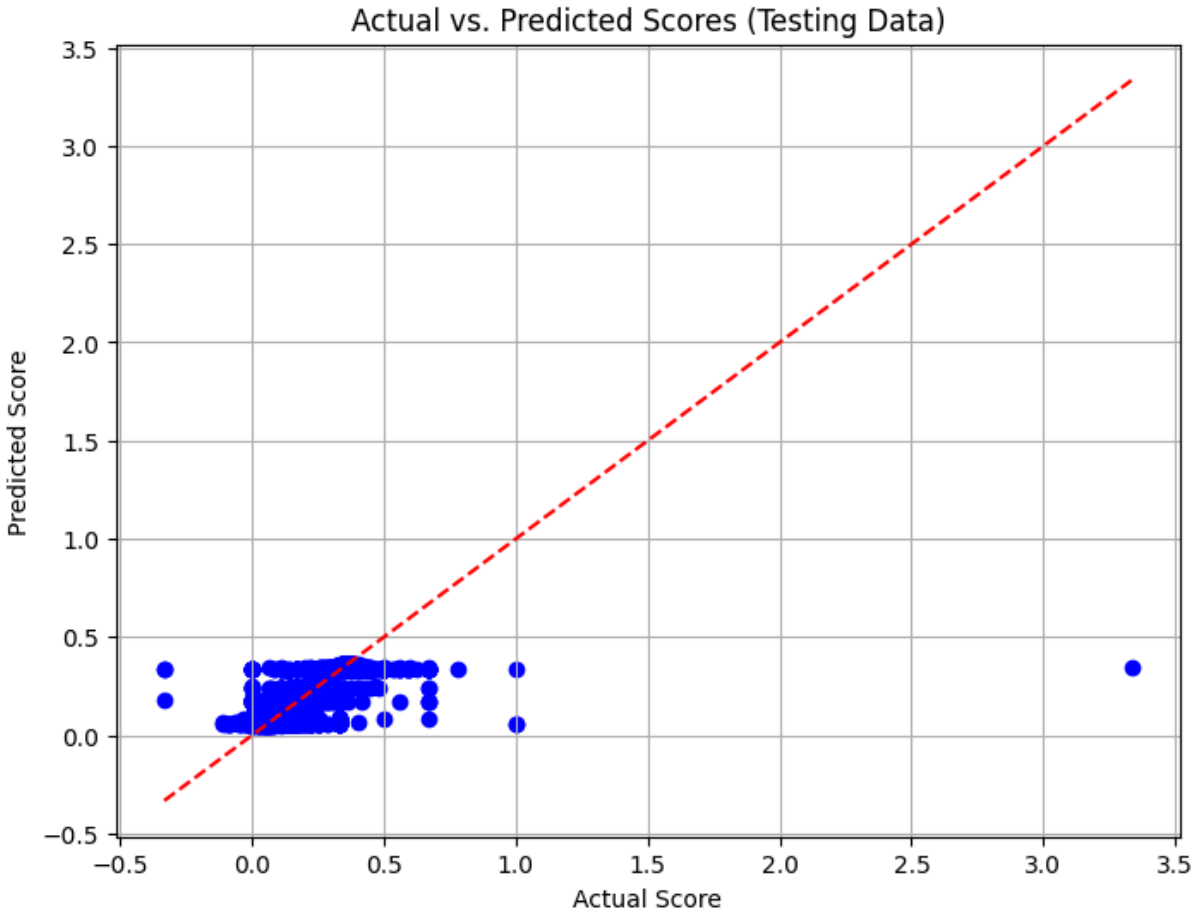
```
combined_data['throws'] = joined_data_copy['throws']
combined_data['bats'] = joined_data_copy['bats']
combined_data['g'] = joined_data_copy['g']

# 4. Create a multiple regression model using 'age', 'position', 'height', '
# Split the data into features (X) and target variable (y)
X = combined_data[['age', 'position', 'height', 'weight', 'double_plays', 't
y = combined_data['score']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

# Define preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('onehot_position', OneHotEncoder(), ['position']),
        ('onehot_throws', OneHotEncoder(), ['throws']),
        ('onehot_bats', OneHotEncoder(), ['bats'])
    ],
    remainder='passthrough'
)

# Create a pipeline with preprocessing and linear regression model
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# Fit the model to the training data
model.fit(X_train, y_train)

# Evaluate the model on the testing data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

# Print model coefficients and evaluation metric
print("Model Coefficients:")
print(model.named_steps['regressor'].coef_)
print("Intercept:", model.named_steps['regressor'].intercept_)
print("Mean Squared Error:", mse)
```

```
Model Coefficients:
[ 1.76538698e-01  9.28097202e-03 -7.59935925e-02  7.99428264e-02
 -7.73362010e-02 -1.00792337e-01 -1.16403664e-02  4.46733407e-04
 -4.46733407e-04  2.45363746e-03 -1.99588068e-03 -4.57756775e-04
  1.54525624e-04 -1.35958175e-04 -9.49824755e-05  1.76227047e-04
 -3.42495650e-07]
Intercept: 0.18611358917206736
Mean Squared Error: 0.004488961271209183
```

In [51]:
```
from sklearn.metrics import r2_score

# Predict the scores for the training data
y_train_pred = model.predict(X_train)

# Calculate the R-squared value for the training data
```

```
r2_train = r2_score(y_train, y_train_pred)

# Predict the scores for the testing data
y_test_pred = model.predict(X_test)

# Calculate the R-squared value for the testing data
r2_test = r2_score(y_test, y_test_pred)

# Print the R-squared values
print("R-squared (Training):", r2_train)
print("R-squared (Testing):", r2_test)

# Visualize the predicted vs. actual scores for the testing data
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_test_pred, color='blue')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red'
plt.xlabel('Actual Score')
plt.ylabel('Predicted Score')
plt.title('Actual vs. Predicted Scores (Testing Data)')
plt.grid(True)
plt.show()
```

```
R-squared (Training): 0.7414718490562193
R-squared (Testing): 0.6894822177065616
```



In [52]:
```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
```

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error

# Change all positions in the 'OF' table to 'OF'
of_data['position'] = 'OF'
c_data.rename(columns={'pos': 'position'}, inplace=True)

# Concatenate the dataframes into one big dataframe
combined_data = pd.concat([c_data, p_data, b1_data, b2_data, b3_data, ss_dat
# Combine all tables into one big table with only the 'age', 'position', and
combined_data = pd.concat([c_data[['age', 'position', 'score']],
                           p_data[['age', 'position', 'score']],
                           b1_data[['age', 'position', 'score']],
                           b2_data[['age', 'position', 'score']],
                           b3_data[['age', 'position', 'score']],
                           ss_data[['age', 'position', 'score']],
                           of_data[['age', 'position', 'score']]])

# Split the data into features (X) and target variable (y)
X = combined_data[['age', 'position']]
y = combined_data['score']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

# Define preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(), ['position'])
    ],
    remainder='passthrough'
)

# Create a pipeline with preprocessing and linear regression model
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# Fit the model to the training data
model.fit(X_train, y_train)

# Evaluate the model on the testing data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

# Print model coefficients and evaluation metric
print("Model Coefficients:")
print(model.named_steps['regressor'].coef_)
print("Intercept:", model.named_steps['regressor'].intercept_)
print("Mean Squared Error:", mse)

# Perform ten-fold cross-validation
try:
    ten_fold_scores = -cross_val_score(
```

```
        model, X, y, scoring='neg_mean_squared_error', cv=10
    )
    print("Ten-Fold Cross-Validation Scores:", ten_fold_scores)
except Exception as e:
    print("Error during ten-fold cross-validation:", e)
```

```
Model Coefficients:
[ 0.17636603  0.01258472 -0.07750667  0.07651909 -0.08024597 -0.10487681
 -0.00284039  0.00023343]
Intercept: 0.15764239107702585
Mean Squared Error: 0.00450618246162656
Ten-Fold Cross-Validation Scores: [       nan 0.00169435 0.00181615 0.001934
38 0.00151659 0.00726391
 0.01075174 0.00470734        nan 0.00125572]
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
ValueError: Found unknown categories ['C'] in column 0 during transform
```

```
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
ValueError: Found unknown categories ['SS'] in column 0 during transform
```

```
 warnings.warn(
```

In [53]:
```
#all cells after this cell is trying to problem solve the NaN
```

In [54]:
```
import pandas as pd



# Check for missing values
missing_values = combined_data.isnull().sum()
print("Missing Values:")
print(missing_values)

# Check for outliers
# For numeric columns like 'age' and 'score'
numeric_cols = ['age', 'score']
outliers = {}
for col in numeric_cols:
    # Calculate the 1st and 3rd quartiles
    q1 = combined_data[col].quantile(0.25)
    q3 = combined_data[col].quantile(0.75)

    # Calculate the interquartile range
    iqr = q3 - q1

    # Define the upper and lower bounds for outliers
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    # Find outliers
    col_outliers = combined_data[(combined_data[col] < lower_bound) | (combi
    outliers[col] = col_outliers

# Print outliers
print("\nOutliers:")
for col, outliers_df in outliers.items():
    print(f"{col} outliers:")
    print(outliers_df)
```

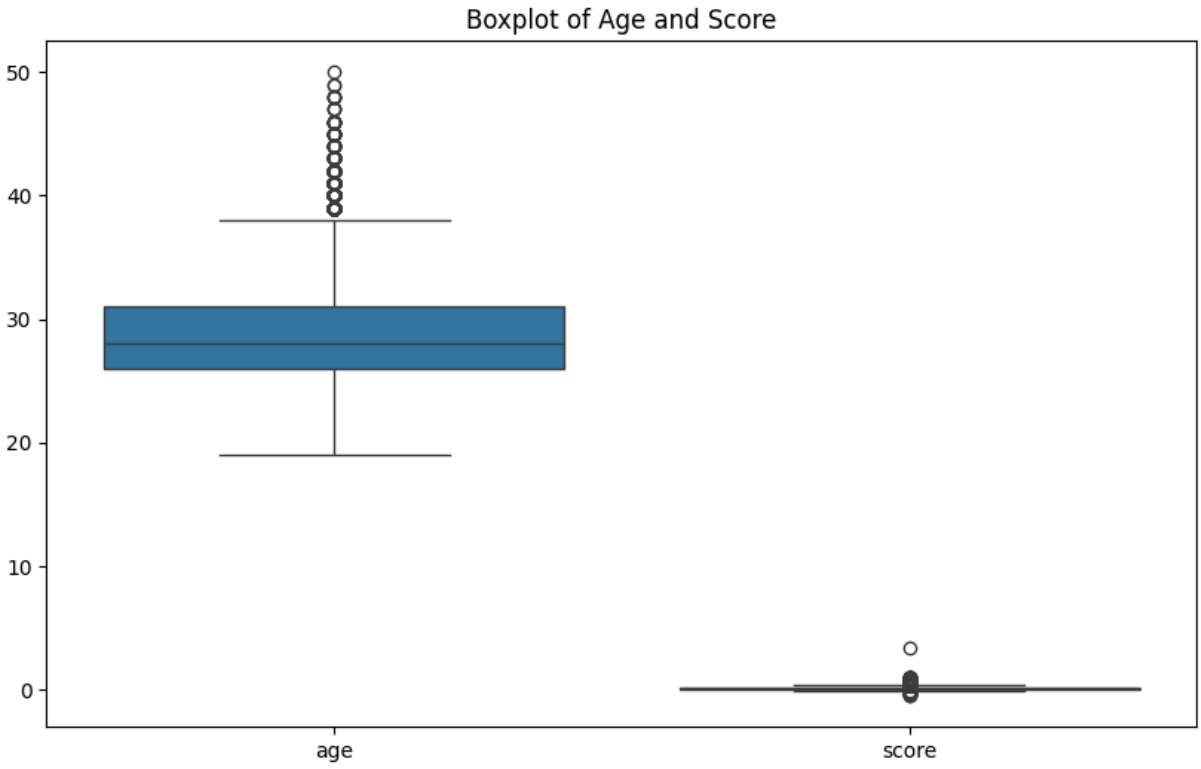
```
Missing Values:
age         0
position    0
score       0
dtype: int64

Outliers:
age outliers:
       age position     score
32486   39        C  0.219178
3568    42        C  0.172524
4707    39        C  0.333333
10846   46        C  0.148541
10845   45        C  0.180760
...    ...      ...       ...
16889   41       OF  0.060976
32299   40       OF  0.062147
32298   39       OF  0.075506
34127   40       OF  0.055556
34126   39       OF  0.060784

[819 rows x 3 columns]
score outliers:
       age position     score
20809   23        C  0.365854
31961   34        C  0.373737
20734   29        C  0.380952
4304    27        C  0.666667
3619    31        C  0.666667
...    ...      ...       ...
28433   33       OF  0.500000
8403    29       OF  0.500000
33018   22       OF  1.000000
12326   31       OF  0.666667
23162   32       OF  0.666667

[1866 rows x 3 columns]
```

In [55]:

```python
# Check for missing values
print("Missing Values in Combined Data:")
print(combined_data.isnull().sum())

# Visualize distribution of numerical features
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.boxplot(data=combined_data[['age', 'score']])
plt.title("Boxplot of Age and Score")
plt.show()

# Check model stability by experimenting with hyperparameters

from sklearn.linear_model import Ridge

alpha_values = [0.1, 1.0, 10.0]  # Different values of alpha (regularizatior
```

```
for alpha in alpha_values:
    ridge_model = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('regressor', Ridge(alpha=alpha))  # Ridge regression with different
    ])
    ten_fold_scores = -cross_val_score(
        ridge_model, X, y, scoring='neg_mean_squared_error', cv=10
    )
    print(f"Ridge Regression (alpha={alpha}):")
    print("Ten-Fold Cross-Validation Scores:", ten_fold_scores)
    print("Mean Score:", ten_fold_scores.mean())
    print()
```

```
Missing Values in Combined Data:
age         0
position    0
score       0
dtype: int64
```



```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
ValueError: Found unknown categories ['C'] in column 0 during transform
```

```
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
ValueError: Found unknown categories ['SS'] in column 0 during transform
```

```
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
```

```
ValueError: Found unknown categories ['C'] in column 0 during transform

  warnings.warn(
Ridge Regression (alpha=0.1):
Ten-Fold Cross-Validation Scores: [       nan 0.00169434 0.00181615 0.001934
38 0.00151659 0.00726391
 0.01075175 0.00470735        nan 0.00125576]
Mean Score: nan

Ridge Regression (alpha=1.0):
Ten-Fold Cross-Validation Scores: [       nan 0.00169433 0.00181616 0.001934
38 0.00151658 0.00726389
 0.01075185 0.00470748        nan 0.00125608]
Mean Score: nan

Ridge Regression (alpha=10.0):
Ten-Fold Cross-Validation Scores: [       nan 0.00169415 0.00181627 0.001934
31 0.00151657 0.00726376
 0.0107537  0.00470907        nan 0.00125944]
Mean Score: nan
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
ValueError: Found unknown categories ['SS'] in column 0 during transform
```

```
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
ValueError: Found unknown categories ['C'] in column 0 during transform
```

```
  warnings.warn(
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
```

```
ValueError: Found unknown categories ['SS'] in column 0 during transform

  warnings.warn(
```

In [56]:

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error

# Define the function to remove outliers using the IQR method
def remove_outliers_iqr(df, column, k=1.5):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - k * IQR
    upper_bound = Q3 + k * IQR
    filtered_df = df[(df[column] >= lower_bound) & (df[column] <= upper_bour
    return filtered_df

# Change all positions in the 'OF' table to 'OF'
of_data['position'] = 'OF'
c_data.rename(columns={'pos': 'position'}, inplace=True)

# Concatenate the dataframes into one big dataframe
combined_data = pd.concat([c_data, p_data, b1_data, b2_data, b3_data, ss_dat

# Remove outliers from the 'score' column using the IQR method
combined_data_cleaned = remove_outliers_iqr(combined_data, 'score')

# Split the cleaned data into features (X) and target variable (y)
X = combined_data_cleaned[['age', 'position']]
y = combined_data_cleaned['score']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

# Define preprocessing steps
preprocessor = ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(), ['position'])
    ],
    remainder='passthrough'
)

# Create a pipeline with preprocessing and linear regression model
model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# Fit the model to the training data
model.fit(X_train, y_train)
```

```
# Evaluate the model on the testing data
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

# Print model coefficients and evaluation metric
print("Model Coefficients:")
print(model.named_steps['regressor'].coef_)
print("Intercept:", model.named_steps['regressor'].intercept_)
print("Mean Squared Error:", mse)

# Perform ten-fold cross-validation on the cleaned data
try:
    ten_fold_scores = -cross_val_score(
        model, X, y, scoring='neg_mean_squared_error', cv=10
    )
    print("Ten-Fold Cross-Validation Scores:", ten_fold_scores)
except Exception as e:
    print("Error during ten-fold cross-validation:", e)
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
ValueError: Found unknown categories ['C'] in column 0 during transform
```

```
  warnings.warn(
Model Coefficients:
[ 0.14339751  0.01769622 -0.06794217  0.07558555 -0.07384286 -0.09782451
  0.00293026  0.00019829]
Intercept: 0.15148030431808282
Mean Squared Error: 0.0019473822210182435
Ten-Fold Cross-Validation Scores: [       nan 0.00152023 0.00134769 0.001488
29 0.00150598 0.00218505
 0.00509761 0.00281817        nan 0.00078048]
```

```
/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-packages/sk
learn/model_selection/_validation.py:1011: UserWarning: Scoring failed. The
score on this train-test partition for these parameters will be set to nan.
Details:
Traceback (most recent call last):
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 137, in __call__
    score = scorer._score(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 345, in _score
    y_pred = method_caller(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/metrics/_scorer.py", line 87, in _cached_call
    result, _ = _get_response_values(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_response.py", line 238, in _get_response_values
    y_pred, pos_label = prediction_method(X), None
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 602, in predict
    Xt = transform.transform(Xt)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 1014, in transform
    Xs = self._call_func_on_transformers(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/compose/_column_transformer.py", line 823, in _call_func_on_tr
ansformers
    return Parallel(n_jobs=self.n_jobs)(jobs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 67, in __call__
    return super().__call__(iterable_with_config)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1918, in __call__
    return output if self.return_generator else list(output)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/joblib/parallel.py", line 1847, in _get_sequential_output
    res = func(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/parallel.py", line 129, in __call__
    return self.function(*args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/pipeline.py", line 1283, in _transform_one
    res = transformer.transform(X, **params.transform)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/utils/_set_output.py", line 295, in wrapped
    data_to_wrap = f(self, X, *args, **kwargs)
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 1023, in transform
    X_int, X_mask = self._transform(
  File "/Users/madisonmontgomery/anaconda3/envs/mynev/lib/python3.9/site-pac
kages/sklearn/preprocessing/_encoders.py", line 213, in _transform
    raise ValueError(msg)
ValueError: Found unknown categories ['SS'] in column 0 during transform
```

```
  warnings.warn(
```

In [ ]: